MYTHOS SYSTEMS

PRISMATIK

ENTERPRISE TRADING INTELLIGENCE PLATFORM

Cornerstone Product, Architecture, Security & Implementation Blueprint

An executive-grade vision and build-ready deep dive for a multi-asset intelligence platform designed to research, validate, simulate, secure, and operationalize next-generation market decisions.

AARON STOVALL

MYTHOS SYSTEMS | 2026 | FOUNDATIONAL BLUEPRINT

FOUNDATIONAL BLUEPRINT

PRISMATIK

Enterprise Trading Intelligence Platform

Executive Product Vision, Granular Architecture, Security Doctrine, and Phased Implementation Plan

CORNERSTONE INTENT This document is designed to function as the durable north-star blueprint for product, engineering, security, quantitative research, operations, partnerships, and future fundraising. It integrates the complete v0.1 product architecture with the build-ready v0.2 evolution contracts without discarding the underlying technical detail.

DOCUMENT CONTROL

DOCUMENT	PRISMATIK Cornerstone Product, Architecture, Security & Implementation Blueprint
ORGANIZATION	Mythos Systems
AUTHOR	Aaron Stovall
YEAR	2026
STATUS	Foundational Blueprint / Executive Architecture Deep Dive
INTEGRATED SOURCES	PRISMATIK Overview v0.1 and Architecture Evolution v0.2
INTENDED AUDIENCE	Founders, engineering leadership, quantitative research, security, product, investors, partners, and future implementation teams

EXECUTIVE THESIS

PRISMATIK is a multi-asset market intelligence operating system built to convert fragmented data into **evidence-backed, probabilistic, reproducible, and risk-aware decisions**. It begins with a powerful crypto intelligence foundation through CoinGecko, expands into U.S. equity, institutional, filing, macro, and options intelligence, and matures into a governed research-to-execution platform.

CORE WEDGE

Event-aware options opportunity intelligence informed by flow, volatility, institutional activity, market regime, and quantitative validation.

PRIMARY FOUNDATION

CoinGecko-powered crypto intelligence delivered through a provider-neutral Rust data gateway with provenance, caching, and rate governance.

TRUST MODEL

Evidence before conclusion, probability instead of prophecy, deterministic controls, explainable models, and secure local or enterprise deployment.

DELIVERY MODEL

A phased journey from crypto MVP to options, quant, simulation, paper trading, controlled execution, enterprise, and a signed ecosystem.

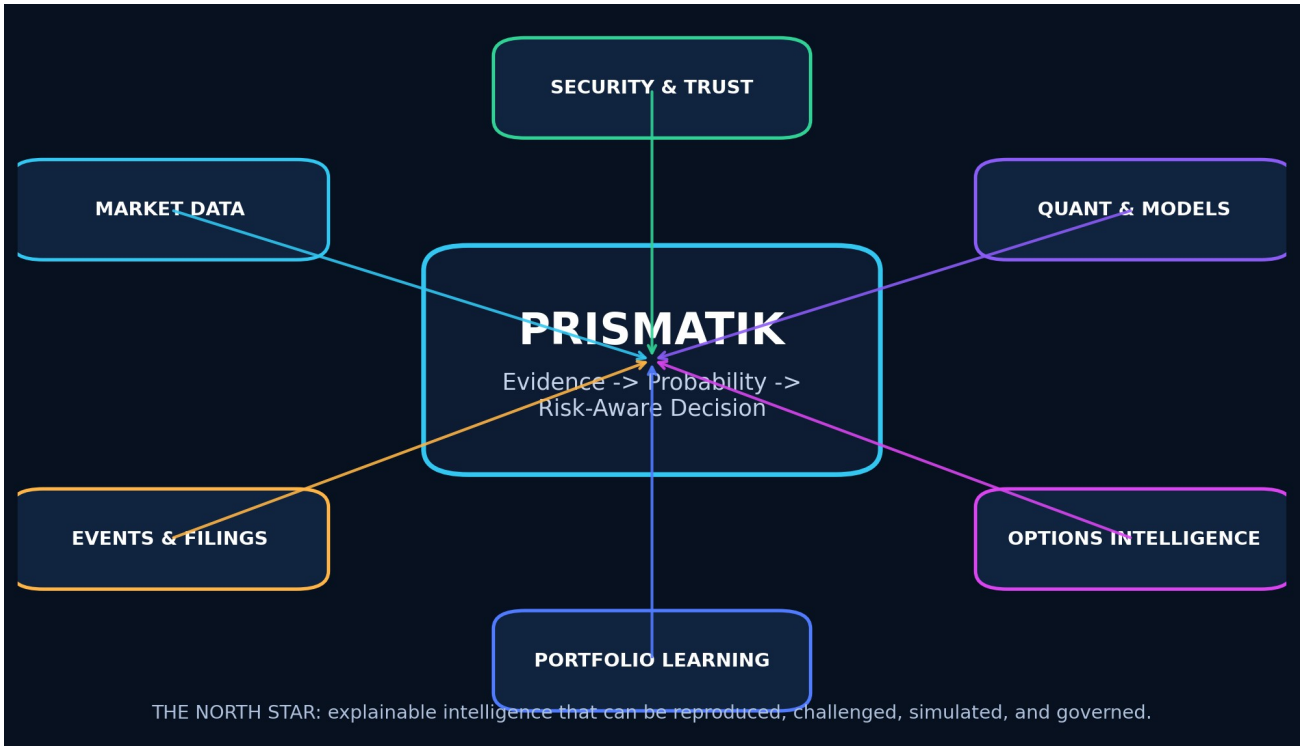

Figure 1 - PRISMATIK north-star system: evidence, quantitative validation, risk, security, and learning converge in one decision platform.

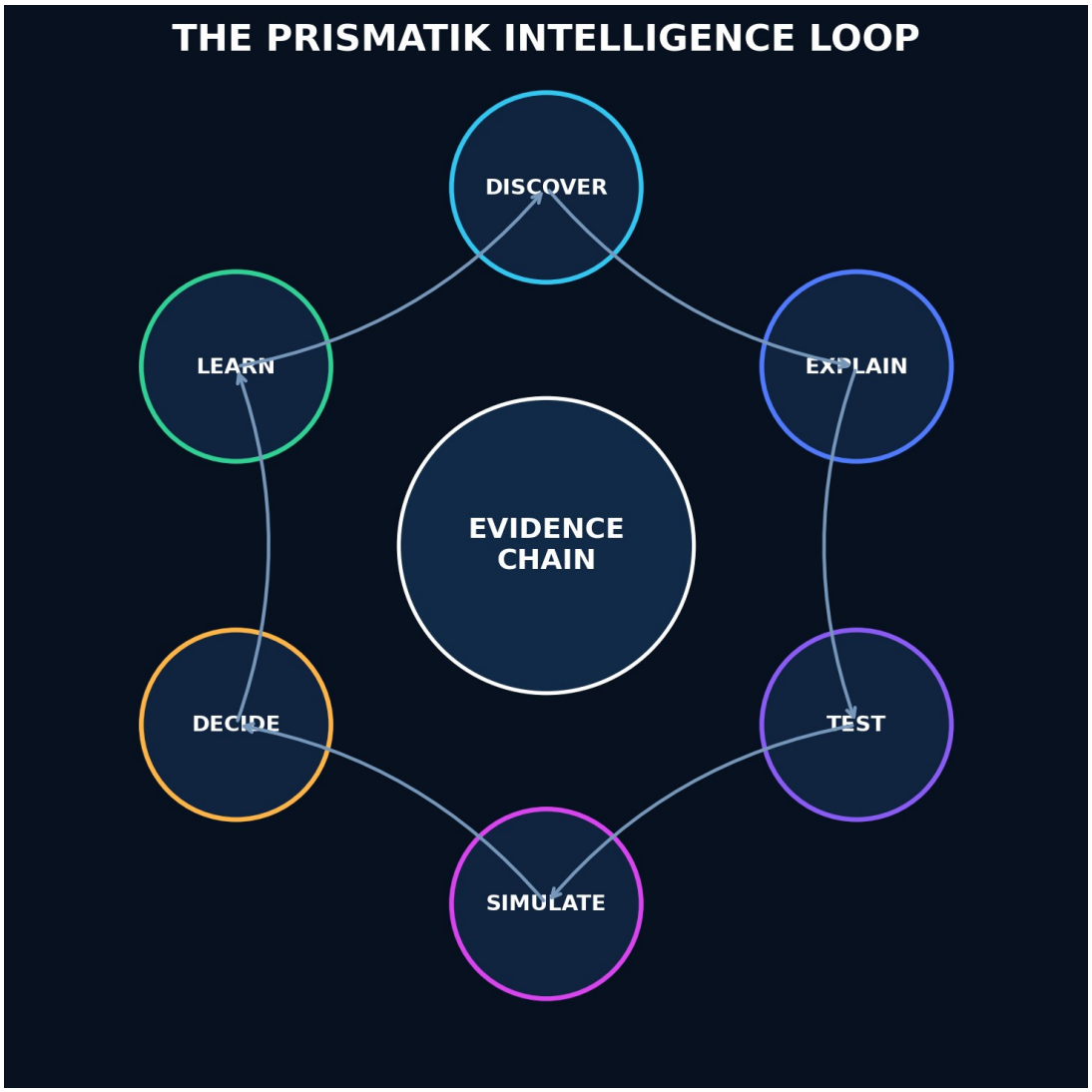

Figure 2 - The PRISMATIK intelligence loop: discover, explain, test, simulate, decide, and learn.

CONTENTS

Table of Contents

The table is generated from Heading 1-3 styles. Microsoft Word may prompt to refresh fields when the document is opened.

DOCUMENT CONVENTIONS

NORMATIVE LANGUAGE MUST indicates a mandatory architectural or engineering requirement. SHOULD identifies the default expectation. MAY indicates an optional capability. NEVER identifies prohibited behavior without explicit senior risk approval.

MUST	Mandatory. A deviation requires an explicit architecture or risk decision.
SHOULD	The expected default. Deviations require a practical reason.
MAY	Optional and acceptable when it increases value without violating stronger requirements.
NEVER	Prohibited unless a senior reviewer explicitly accepts and records the risk.
Evidence language	Forecasts, scores, and signals are estimates with provenance and uncertainty - never guarantees.

How to Use This Report

- Executives and partners should begin with the Executive Thesis, Mission, Capability Map, System Architecture, Security Doctrine, and Roadmap.
- Product and design teams should focus on the Experience and Visual Design System, Detailed Feature Design, Opportunity Engine, AI Analyst, and workspace model.
- Engineering teams should treat the provider ports, canonical domain model, data pipeline, storage, streaming, trait surfaces, and repository boundaries as implementation contracts.
- Quantitative research teams should focus on the strategy runtime, backtesting, validation, simulation, model serving, feature store, and reproducibility manifest.
- Security and operations teams should treat the security, cryptographic agility, on-premises deployment, observability, supply-chain, backup, and release sections as foundational controls.

MYTHOS SYSTEMS / PRISMATIK

01

STRATEGIC FOUNDATION

Mission, market wedge, personas, principles, product scope, and the executive thesis for PRISMATIK.

CORNERSTONE EXECUTIVE ARCHITECTURE REPORT | 2026

1. Executive Summary

PRISMATIK is an advanced, multi-asset trading intelligence platform centered on **decision quality**, not unsupported promises of guaranteed returns. Its purpose is to transform fragmented market information into structured, testable, risk-aware trade hypotheses.

The solution converges:

- U.S. equity and ETF intelligence.
- Listed options flow, chain analytics, volatility analysis, and strategy construction.
- Cryptocurrency market, token, exchange, category, derivative, and on-chain-adjacent intelligence.
- SEC filing intelligence, including institutional holdings, insider transactions, ownership disclosures, and material corporate events.
- CFTC Commitments of Traders positioning.
- Macroeconomic data and event awareness.
- Quantitative strategy definition, backtesting, validation, optimization, and Monte Carlo simulation.
- Explainable probabilistic forecasts.
- Portfolio risk, scenario analysis, journaling, and post-trade learning.
- AI-assisted investigation through controlled tools and signed MCP integrations.
- A premium desktop interface with refined animation, soft spectral glows, GPU-accelerated visualizations, and dense professional workflows.

PRISMATIK MUST not represent probabilistic model output as certainty. Every signal, forecast, and recommended structure MUST show:

1. The evidence used.
2. Data freshness and provider provenance.
3. Model or rule version.
4. Confidence and uncertainty.
5. Known blind spots.
6. Historical validation context.
7. Liquidity and execution constraints.
8. Risk of loss and invalidation conditions.

The platform begins as a **modular monolith** with hard architectural boundaries, not an early microservice maze. Rust owns the trusted core, ingestion, quantitative computation, security, persistence, orchestration, and native integrations. SvelteKit/Svelte 5 and TypeScript own the presentation layer and interaction model. Tauri 2 delivers the secure desktop shell. WebGPU/wgpu powers visualizations and selected compute workloads where profiling proves value.

2. Product Mission, Positioning, and Guardrails

2.1 Mission

> **DESIGN PRINCIPLE** Help serious traders discover meaningful market setups, understand why they matter, test whether the thesis has historically held, choose an appropriate trade structure, quantify possible outcomes, and learn from actual results.

2.2 Positioning

PRISMATIK is not merely:

- A charting terminal.
- A brokerage client.
- An options-flow feed.
- A cryptocurrency tracker.
- A black-box “AI picks” product.
- A social trading feed.
- A strategy backtester.
- A filing reader.

It is a **market intelligence and quantitative decision operating system** that integrates all of those functions behind one typed, explainable, provider-independent architecture.

2.3 Initial Market Wedge

The first defensible wedge SHOULD be:

> **DESIGN PRINCIPLE** Event-aware options opportunity intelligence informed by flow, volatility, institutional activity, technical structure, macro regime, and quantitative validation.

This wedge is focused enough to produce differentiated value while the broader multi-asset platform matures.

2.4 Product Guardrails

PRISMATIK MUST:

- Distinguish research, simulation, paper trading, alerts, and live execution.
- Default new accounts to research and paper-trading mode.
- Require explicit enablement and step-up authorization for live execution.
- Clearly mark delayed, derived, estimated, and incomplete data.
- Never invent unavailable exchange, options, filing, or market data.
- Preserve source provenance down to the provider endpoint and retrieval timestamp.
- Detect stale feeds and fail closed for automation that depends on timely data.
- Avoid dark patterns that encourage overtrading.
- Support risk limits that cannot be bypassed casually.
- Keep AI tool permissions narrowly scoped.
- Record immutable audit events for sensitive operations.
- Make all automated actions cancellable, bounded, idempotent, and observable.

3. User Personas and Primary Jobs to Be Done

3.1 Advanced Retail Options Trader

Needs to:

- Detect unusual options activity.
- Determine whether activity is directional, hedging, closing, or part of a spread.
- Compare implied movement with historical movement.
- Select a structure with acceptable liquidity, break-even, theta, and volatility exposure.
- Understand upcoming catalysts.
- Test similar setups.
- Receive alerts when thesis conditions align or invalidate.

3.2 Quantitative Researcher

Needs to:

- Create strategies using a visual DSL, SQL, Python, or Rust.
- Run reproducible backtests.
- Control survivorship, look-ahead, and corporate-action handling.
- Run walk-forward and out-of-sample validation.
- Compare parameter stability.
- Simulate portfolio-level risk.
- Export notebooks, datasets, and signed result manifests.

3.3 Crypto Trader

Needs to:

- View broad market and category conditions.
- Analyze price, volume, market capitalization, liquidity, exchanges, and historical OHLCV.
- Track trending assets, categories, and market dominance.
- Compare centralized and decentralized market information.
- Monitor derivatives and exchange-native data through optional adapters.
- Analyze cross-asset correlation and regime effects.

3.4 Long-Term Investor

Needs to:

- Follow institutional ownership changes.
- Read and summarize filings.
- Track insiders and major holders.
- Understand portfolio concentration, factor exposure, and drawdown risk.
- Separate short-term noise from long-term thesis changes.

3.5 Risk Manager or Trading Team Lead

Needs to:

- Define account and strategy limits.
- Review aggregate exposures and correlated risk.
- Enforce approval workflows.

- Audit model changes and automated actions.
- Run stress scenarios and emergency stop procedures.

3.6 Platform Administrator

Needs to:

- Configure providers, credentials, entitlements, retention, and deployment.
- Monitor health, rate limits, costs, errors, and latency.
- Enforce RBAC/ABAC policies.
- Rotate secrets and cryptographic material.
- Manage signed plugins and MCP servers.
- Export compliance and audit records.

4. Product Principles

4.1 Evidence Before Conclusion

Every opportunity MUST expose the evidence chain that produced it.

4.2 Probability, Not Prophecy

Model output MUST be expressed as a distribution, interval, calibrated probability, ranking, or scenario set—not a guaranteed prediction.

4.3 Provider Independence

Business logic MUST depend on domain ports, never directly on a vendor SDK or endpoint.

4.4 Local-First Where Valuable

Watchlists, layouts, journals, cached research, model metadata, and selected datasets SHOULD remain available offline. Cloud sync is optional and encrypted.

4.5 Security as Architecture

Secrets, execution credentials, personal financial data, strategy intellectual property, model artifacts, and audit logs are high-value assets.

4.6 Professional Density Without Confusion

The interface can be information-rich, but every screen MUST support progressive disclosure, keyboard navigation, saved layouts, and clear provenance.

4.7 Reproducibility

A backtest or forecast MUST be reproducible from:

- Dataset snapshot.
- Data-cleaning policy.
- Strategy/model version.
- Parameters.
- Random seed.
- Execution assumptions.
- Code and dependency versions.

4.8 Graceful Degradation

If advanced GPU rendering, a provider, AI model, or streaming feed is unavailable, the application MUST preserve core workflows in a reduced-capability mode.

5. Scope and Asset Coverage

5.1 Phase-One Asset Classes

- U.S. common stocks.
- U.S. ETFs.
- Listed equity and ETF options.
- Cryptocurrency spot-market intelligence.
- Cryptocurrency reference, category, exchange, and historical data.
- Macro instruments relevant to risk regime analysis.
- Futures-positioning intelligence from CFTC COT reports.

5.2 Later Asset Classes

- Crypto perpetual futures and options through exchange-specific adapters.
- U.S. index options.
- Futures and options on futures.
- Foreign exchange.
- Fixed income and yield curves.
- Commodities.
- Prediction markets.
- International equities where licensing permits.

5.3 Data Rights and Entitlements

The platform MUST maintain a data-entitlement registry that records:

- Provider.
- Plan/tier.
- Licensed use.
- Redistribution restrictions.
- Display rights.
- Historical retention rights.
- Derived-data rights.
- User or organization scope.
- Expiration.
- Required attribution.
- Audit owner.

No endpoint integration is production-ready until legal and licensing constraints are documented.

6. Experience and Visual Design System

6.1 Design Vision

The visual language should feel like a fusion of:

- A professional trading terminal.
- A high-end scientific instrument.
- A cinematic observatory.
- A modern security operations center.
- A restrained, premium, spectral interface.

The interface MUST be captivating without undermining readability or making motion distracting.

6.2 Visual Characteristics

- Deep graphite, obsidian, and midnight surfaces.
- Layered translucent panels with subtle depth.
- Soft spectral glows rather than harsh neon borders.
- Prismatic accents that change by domain:
  - Cyan: data and neutral intelligence.
  - Emerald: favorable or positive conditions.
  - Amber: uncertainty, catalyst proximity, or caution.
  - Magenta/violet: AI and model-derived intelligence.
  - Red: actual risk, failure, or invalidation—not decoration.
- Fine grid textures and subtle star-field/noise effects at low opacity.
- Responsive light refraction on focus changes.
- Controlled bloom around high-value active elements.
- Smooth panel docking and tab transitions.
- GPU-rendered heatmaps, risk surfaces, network graphs, and time-series layers.
- Reduced-motion and high-contrast modes.

6.3 Motion System

Motion MUST communicate state.

Approved motion categories:

9. **Focus motion:** subtle glow and 100–180 ms elevation.
10. **Context transition:** 180–280 ms crossfade/slide with preserved spatial orientation.
11. **Streaming pulse:** low-amplitude pulse when fresh data arrives.
12. **Alert emergence:** staged reveal that communicates severity without flashing.
13. **Simulation playback:** timeline scrub, scenario paths, and distribution evolution.
14. **Execution state:** explicit pending, acknowledged, partially filled, filled, cancelled, or rejected animation.
15. **Background ambience:** optional, GPU-cheap, slow, and disabled by reduced-motion settings.

Animation budgets MUST be measured. No animation may block input, delay risk information, or reduce chart accuracy.

6.4 Workspace Model

The application SHOULD support:

- Dockable panels.
- Resizable regions.
- Multi-monitor workspaces.
- Named layouts.

- Per-strategy workspaces.
- Keyboard command palette.
- Quick symbol switcher.
- Linked panels by symbol, timeframe, strategy, or account.
- Pop-out native windows.
- Layout snapshots and restore.
- Role-based default workspaces.

6.5 Primary Navigation

Recommended sections:

- **Command**
- **Markets**
- **Options**
- **Crypto**
- **Institutions**
- **Events**
- **Research**
- **Strategies**
- **Simulations**
- **Portfolio**
- **Journal**
- **Automations**
- **AI Analyst**
- **Data**
- **Administration**

7. Functional Capability Map

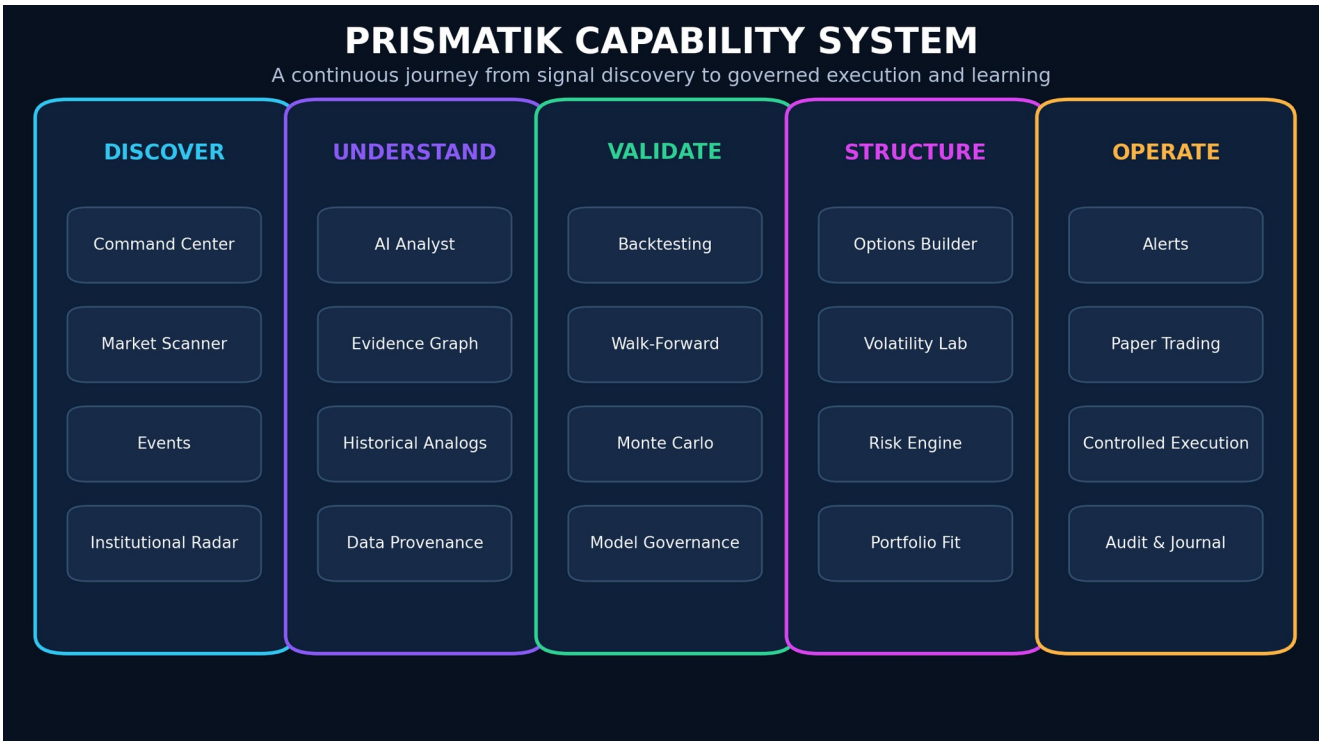

Figure 3 - PRISMATIK capability system from discovery through governed operation.

7.1 Market Command Center

- Global market state.
- U.S. equity index overview.
- Sector and industry performance.
- Crypto market overview.
- Volatility and breadth.
- Macro calendar.
- Event risk.
- Watchlists.
- Top opportunities.
- Portfolio alerts.
- Feed health and freshness.

7.2 Opportunity Engine

- Candidate generation.
- Rule/model scoring.
- Evidence fusion.
- Liquidity filtering.
- Event-aware ranking.
- Risk-adjusted ranking.
- Duplicate thesis collapse.
- User-specific suitability filters.
- Explainable opportunity cards.

7.3 Options Intelligence

- Chains.
- Greeks.
- Open interest and volume.
- Flow.
- Sweeps and blocks.
- Volatility surface.
- Skew and term structure.
- Implied move.
- Dealer-exposure estimates.
- Strategy builder.
- Scenario P/L.
- Monte Carlo distribution.
- Liquidity and fill-quality scoring.

7.4 Institutional Intelligence

- 13F holdings and deltas.
- 13D/13G ownership.
- Form 4 insider activity.
- 8-K event extraction.
- CFTC COT positioning.
- Congressional data where legally and contractually available.
- Dark-pool and block activity where available.
- Institution/company relationship graph.

7.5 Crypto Intelligence

- Prices and market data.
- Categories and narratives.
- Market dominance.
- Trending assets.
- Exchange metadata.
- Historical market charts.
- OHLC.
- Token metadata.
- Derivative summaries where provider coverage allows.
- DEX/pool intelligence where covered.
- Optional exchange-native order-book adapters.

7.6 Research and Quant Lab

- Visual rule builder.
- Strategy DSL.
- Python research environment.
- Rust strategy SDK.
- SQL analytics.
- Backtesting.
- Walk-forward validation.
- Monte Carlo.
- Parameter sensitivity.
- Regime analysis.
- Portfolio simulation.
- Reproducibility manifests.

7.7 Risk and Portfolio

- Positions and transactions.
- Exposure.
- Greeks.
- Concentration.
- Correlation.
- Factor exposure.
- Drawdown.
- Stress tests.
- VaR/CVaR as supplemental measures.
- Scenario analysis.
- Pre-trade guardrails.
- Portfolio kill switches.

7.8 Journal and Learning

- Automatic trade capture.
- Manual thesis.
- Screenshots and chart state.
- Emotional/behavior tags.
- Planned versus actual execution.
- Post-trade review.
- Mistake taxonomy.
- AI pattern discovery.
- Strategy-specific scorecards.

7.9 Automation

- Alerts.
- Scheduled scans.
- Data refresh.
- Strategy runs.
- Paper execution.
- Conditional live execution in later phases.
- Human approval gates.
- Emergency disablement.
- Full auditability.

MYTHOS SYSTEMS / PRISMATIK

02

PRODUCT EXPERIENCE & CAPABILITY SYSTEM

The captivating interface, functional map, intelligence workflows, and feature-by-feature design.

CORNERSTONE EXECUTIVE ARCHITECTURE REPORT | 2026

8. Detailed Feature Design

8.1 Command Center

FUNCTIONAL DESIGN

The Command Center is a personalized, role-aware operational overview. It MUST not become an unstructured wall of widgets.

Recommended regions:

- **Market regime:** trend, volatility, liquidity, breadth, correlation, macro risk.
- **Today's catalysts:** earnings, macro releases, filings, options expirations, token events.
- **Priority opportunities:** ranked by expected value, confidence, liquidity, and user strategy fit.
- **Risk posture:** account exposure, portfolio Greeks, drawdown, concentration, upcoming event exposure.
- **Institutional activity:** meaningful ownership and positioning changes.
- **Crypto state:** market cap, volume, dominance, category movement, exchange anomalies.
- **System status:** provider freshness, API budget, queue lag, model status.

Each card MUST include:

- Timestamp.
- Source.
- Freshness state.
- Data quality grade.
- Confidence.
- Click-through evidence.
- Dismiss/snooze behavior.
- Personalization reason.

ADVANCED ENHANCEMENT

Add an **attention governor** that limits repeated low-value alerts and prioritizes information based on:

- Portfolio exposure.
- Watchlist relevance.
- User strategy.
- Event proximity.
- Severity.
- Novelty.
- Historical usefulness.

8.2 Universal Instrument Workspace

Each asset page SHOULD contain linked modules:

- Price chart.
- Market metadata.
- News/events.
- Technical state.
- Options state.
- Institutional state.
- Filing state.
- Crypto-specific token/exchange data.
- AI thesis.
- Historical analogs.
- Strategy candidates.
- Journal history.
- Alerts and automations.

A symbol context object SHOULD be passed between panels, not reconstructed independently.

8.3 Advanced Charting

REQUIRED CAPABILITIES

- Candlestick, bar, line, area.
- Heikin-Ashi.
- Volume and volume profile.
- VWAP and anchored VWAP.
- Corporate-action-aware history.
- Multi-timeframe overlays.
- Compare mode.
- Drawing tools.
- Saved annotations.
- Event markers.
- Filing and institutional markers.
- Options-flow markers.
- Crypto market and category overlays.
- Replay mode.
- Synchronized crosshair.
- GPU-accelerated rendering for large series.

CHART DATA INTEGRITY

Every point MUST preserve:

- Provider.
- Venue or aggregation scope.
- Timestamp resolution.
- Timezone.
- Adjustment policy.
- Currency.
- Missing-data state.
- Data version.

The chart MUST visually distinguish:

- Live.
- Delayed.
- Derived.
- Interpolated.
- Backfilled.
- Stale.

8.4 Market Scanner

The scanner MUST support:

- Saved scans.
- Streaming updates.
- Compound conditions.
- Explainable scoring.
- Preview counts.
- Historical hit rate.
- Alert conversion.
- Export.
- Strategy handoff.

Condition groups:

- Price/return.
- Volume and relative volume.
- Volatility.
- Technical indicators.

- Fundamental metrics.
- Filing activity.
- Institutional activity.
- Options activity.
- Event proximity.
- Market regime.
- Crypto category/exchange metrics.
- User-defined factors.

8.5 Event and Catalyst Engine

The event engine normalizes all relevant events into a common schema.

Examples:

- Earnings.
- Guidance.
- SEC filing.
- Insider transaction.
- Ownership disclosure.
- CFTC report.
- Federal Reserve decision.
- Inflation and employment release.
- FDA or regulatory event.
- Merger or acquisition.
- Product event.
- Dividend.
- Split.
- Options expiration.
- Token unlock.
- Network upgrade.
- Exchange listing/delisting.
- Governance vote.

Every event SHOULD have:

```text
event_id
event_type
source
source_reference
entities
symbols
scheduled_at
observed_at
effective_at
severity
confidence
expected_impact_window
historical_impact_distribution
user_exposure
freshness
raw_document_hash
parser_version
```

The event engine MUST support revisions and corrections without overwriting prior facts.

8.6 Opportunity Card

An opportunity is not a trade instruction. It is a ranked hypothesis.

Recommended fields:

- Asset and direction.
- Suggested expression.
- Opportunity score.
- Expected-value estimate.

- Confidence interval.
- Time horizon.
- Catalyst.
- Evidence summary.
- Contradicting evidence.
- Liquidity score.
- Volatility state.
- Risk and invalidation.
- Similar historical samples.
- Backtest status.
- Simulation status.
- Data freshness.
- Model/rule version.
- “Why shown to me.”
- “What would change this score.”

8.7 Historical Analog Engine

The analog engine finds prior periods with similar:

- Trend.
- Volatility.
- Breadth.
- Macro state.
- Options state.
- Event type.
- Institutional positioning.
- Cross-asset correlations.

It MUST expose the distance metric and avoid presenting a small or biased sample as authoritative.

8.8 Alerting

Alerts can target:

- Price.
- Technical condition.
- Volume.
- Options flow.
- Implied volatility.
- Filing.
- Institutional position.
- COT extreme.
- Event proximity.
- Strategy condition.
- Portfolio risk.
- Data health.
- Model drift.

Alert execution MUST be idempotent and use deduplication windows. Delivery channels may include:

- Desktop.
- Mobile companion.
- Email.
- SMS through an optional provider.
- Slack/Teams.
- Webhook.
- MCP/agent event.
- PagerDuty for enterprise operational alerts.

MYTHOS SYSTEMS / PRISMATIK

03

DATA, PROVIDERS & EVIDENCE ARCHITECTURE

Provider independence, CoinGecko, Unusual Whales, SEC, CFTC, FRED, data quality, lineage, and cost governance.

CORNERSTONE EXECUTIVE ARCHITECTURE REPORT | 2026

9. Data Provider and API Strategy

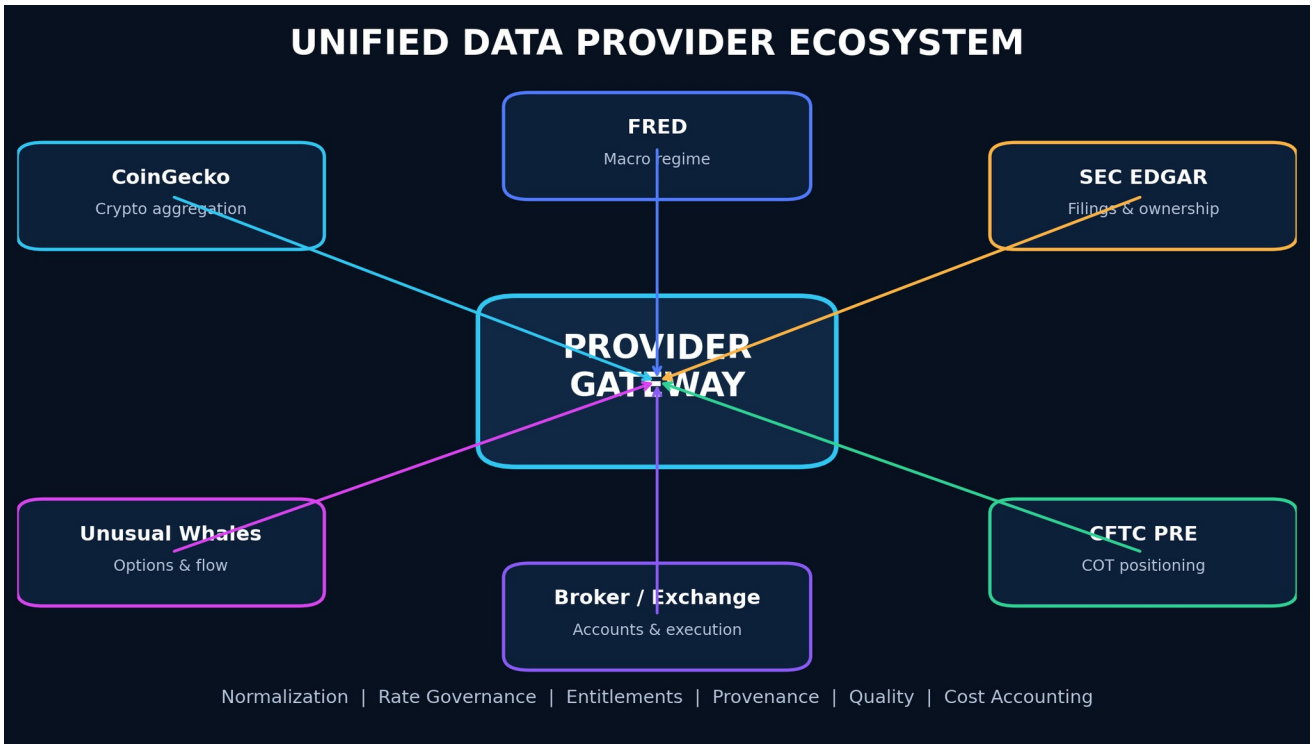

Figure 4 - Provider ecosystem behind a normalized, entitlement-aware gateway.

9.1 Provider Philosophy

No single provider should define the platform's domain model. Adapters MUST normalize provider data into canonical contracts.

Each adapter MUST implement:

- Authentication.
- Rate limiting.
- Pagination.
- Retries with capped exponential backoff and jitter.
- Circuit breaking.
- Freshness tracking.
- Provenance.
- Schema validation.
- Contract tests.
- Cost accounting.
- Entitlement enforcement.
- Provider-specific error translation.
- Health checks.

9.2 CoinGecko — Primary Crypto Aggregation Provider

INTENDED USES

CoinGecko should serve as the primary cryptocurrency reference and broad market data provider for:

- Coin identifiers and metadata.
- Current prices.
- Market capitalization.

- Volume.
- Market rankings.
- Categories.
- Trending data.
- Exchanges.
- Historical market charts.
- OHLC.
- Global crypto market data.
- Derivative summaries when available by endpoint/tier.
- On-chain/GeckoTerminal-related market and pool discovery where licensed.
- News where plan access permits.
- Token and asset discovery.

ARCHITECTURE REQUIREMENTS

- Store CoinGecko **coin IDs** as provider identifiers; never rely on ticker symbols as unique keys.
- Maintain a canonical asset identity service that maps symbols, contracts, chains, exchange pairs, and provider IDs.
- Keep API keys exclusively in the Rust backend or server-side infrastructure.
- Add a request-budget service that tracks rate limits and credits.
- Use cache policies by endpoint class:
  - Metadata: long TTL.
  - Market snapshots: short TTL.
  - Historical candles: immutable segments after finalization.
  - Trending data: moderate TTL.
- Introduce stale-while-revalidate behavior for non-critical UI reads.
- Do not use a demo/free endpoint as the sole source for latency-sensitive production trading.
- Add optional exchange-native adapters for order books, trades, private account data, and execution because an aggregator is not a substitute for venue-native execution data.

COINGECKO FUNCTIONAL MODULES

- Crypto market overview.
- Asset search.
- Asset profile.
- Categories and narratives.
- Global market state.
- Exchange explorer.
- Historical charts.
- Cross-asset correlations.
- Portfolio valuation.
- Market dominance.
- Trend/ranking scanner.
- Token watchlists.
- Data provenance inspector.

9.3 Unusual Whales — Equity and Options Intelligence

INTENDED USES

Subject to plan and licensing:

- Options flow.
- Options chains and derived data.
- Dark-pool data.
- Open interest.
- Greek exposure.
- Volatility.
- Institutional and congressional datasets.
- Insider data.
- Market and ticker analytics.

- MCP-based AI access for approved use cases.

DESIGN REQUIREMENTS

- Prefer REST/OpenAPI-generated clients for deterministic production workflows.
- Treat MCP as an AI tool interface, not the canonical ingestion pipeline.
- Preserve raw provider payloads for replay only when licensing permits.
- Build field-level provenance for all derived scores.
- Separate “trade observed” from “trade interpretation.”
- Explicitly model uncertainty around opening/closing and directional inference.
- Add a provider kill switch if schemas or entitlements change.

9.4 SEC EDGAR — Regulatory and Filing Intelligence

INTENDED USES

- Company submissions history.
- Filing metadata.
- XBRL company facts.
- 10-K.
- 10-Q.
- 8-K.
- 13F.
- 13D/13G.
- Forms 3, 4, and 5.
- Form 144.
- Filing documents and exhibits where permitted.
- RSS or polling-based recent filing discovery.

DESIGN REQUIREMENTS

- Respect SEC fair-access requirements and identify the application appropriately.
- Cache immutable filings.
- Hash original documents.
- Store parser and extraction version.
- Preserve raw text separately from AI-generated summaries.
- Build deterministic parsers before relying on language models.
- Extract entities, holdings, changes, risks, and material events.
- Provide links back to original filings.
- Reprocess historical filings when extraction improves, while retaining prior outputs.

9.5 CFTC Public Reporting Environment — Commitments of Traders

INTENDED USES

- Legacy COT.
- Disaggregated COT.
- Traders in Financial Futures.
- Futures-only.
- Futures-and-options combined.
- Historical positioning.
- Managed-money, dealer, asset-manager, leveraged-fund, producer, and other category analysis as supported by report type.

FEATURE DESIGN

- Normalize contracts and report categories.
- Calculate net positioning.
- Weekly change.
- Z-score.

- Historical percentile.
- Crowding score.
- Price-position divergence.
- Reversal alerts.
- Cross-market positioning map.
- Regime overlays.

COT data is weekly and delayed relative to the reporting date; the interface MUST communicate this clearly.

9.6 FRED — Macroeconomic Data

Recommended uses:

- Interest rates.
- Yield curve.
- Inflation.
- Employment.
- Liquidity and monetary aggregates.
- Credit conditions.
- Financial stress.
- Economic regime classification.

Macro series MUST retain revision/vintage awareness where practical to prevent look-ahead bias.

9.7 Additional Recommended Providers

PUBLIC OR GOVERNMENT

- U.S. Treasury fiscal and yield data.
- Bureau of Labor Statistics.
- Bureau of Economic Analysis.
- Federal Reserve calendars and releases.
- FINRA public datasets where applicable.
- OCC and exchange calendars where permitted.

OPTIONAL PAID/COMMERCIAL

- A licensed consolidated equity/option quote provider.
- Exchange-grade historical tick/options data.
- Corporate actions and reference data.
- News with machine-readable entitlements.
- Broker APIs for accounts and execution.
- Crypto exchange-native feeds.
- Institutional on-chain analytics.

CRITICAL PRINCIPLE

PRISMATIK MUST display the distinction between:

- Official public data.
- Licensed commercial data.
- User-connected broker data.
- Exchange-native data.
- Model-derived data.
- Community-contributed data.

10. Unified Market Data Architecture

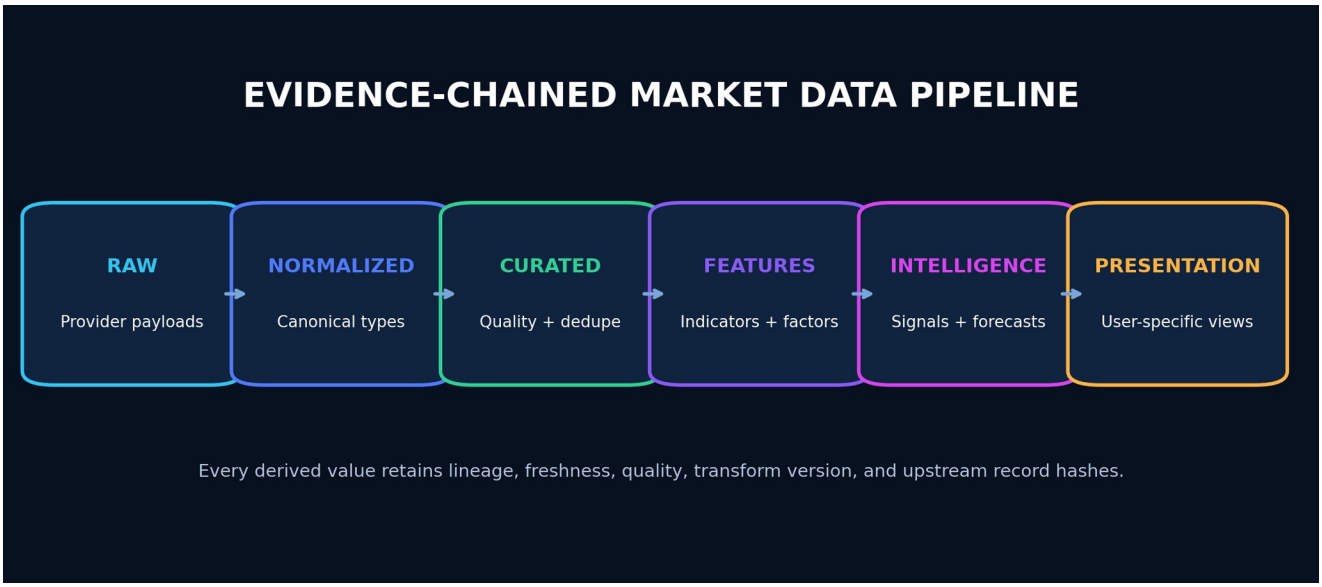

Figure 5 - Evidence-chained market-data pipeline and intelligence layers.

10.1 Canonical Data Layers

16. **Raw layer:** provider payloads, immutable where licensing permits.
17. **Normalized layer:** canonical typed records.
18. **Curated layer:** cleaned, deduplicated, adjusted, and quality-scored.
19. **Feature layer:** indicators, factors, model features.
20. **Intelligence layer:** opportunities, forecasts, anomalies, explanations.
21. **Presentation layer:** user-specific views and aggregations.

10.2 Provider Port Example

```rust
#[async_trait::async_trait]
pub trait MarketDataProvider: Send + Sync {
    fn provider_id(&self) -> ProviderId;

    async fn health(
        &self,
        ctx: &RequestContext,
    ) -> Result<ProviderHealth, ProviderError>;

    async fn resolve_assets(
        &self,
        ctx: &RequestContext,
        query: AssetQuery,
    ) -> Result<Vec<ProviderAsset>, ProviderError>;

    async fn market_snapshot(
        &self,
        ctx: &RequestContext,
        request: MarketSnapshotRequest,
    ) -> Result<MarketSnapshot, ProviderError>;

    async fn historical_bars(
        &self,
        ctx: &RequestContext,
        request: HistoricalBarsRequest,
    ) -> Result<HistoricalBars, ProviderError>;
}
```

Provider capabilities MUST be discoverable rather than assumed.

10.3 Canonical Identity Service

The system MUST solve symbol ambiguity.

Canonical asset records SHOULD include:

- Internal immutable ID.
- Asset type.
- Legal issuer/entity.
- Display symbol.
- Exchange/venue.
- Currency.
- FIGI/CUSIP/ISIN when licensed and applicable.
- SEC CIK.
- CoinGecko ID.
- Chain.
- Contract address.
- Token decimals.
- Provider aliases.
- Valid-from and valid-to dates.
- Corporate-action lineage.

10.4 Data Quality Score

Each dataset and record SHOULD be scored using:

- Freshness.
- Completeness.
- Consistency.
- Provider confidence.
- Cross-provider agreement.
- Schema validity.
- Outlier status.
- Adjustment status.
- Venue coverage.

Low-quality data MUST not silently drive high-confidence recommendations.

10.5 Rate-Limit and Budget Governor

A centralized governor MUST:

- Track provider quotas.
- Reserve capacity for critical workflows.
- Coalesce duplicate requests.
- Cache responses.
- Apply adaptive polling.
- Shed low-priority load.
- Respect `Retry-After`.
- Record spend and credits.
- Alert before exhaustion.
- Prevent N parallel UI components from independently calling the same endpoint.

MYTHOS SYSTEMS / PRISMATIK

04

QUANT, OPTIONS & PREDICTIVE INTELLIGENCE

Strategy research, backtesting, Monte Carlo, model governance, options flow, volatility, and opportunity ranking.

CORNERSTONE EXECUTIVE ARCHITECTURE REPORT | 2026

11. Quantitative Research and Simulation Architecture

11.1 Strategy Definition Modes

VISUAL BUILDER

For non-programmers:

- Condition groups.
- Indicators.
- Events.
- Position rules.
- Option-selection rules.
- Entry/exit.
- Risk limits.
- Time windows.
- Execution assumptions.

STRATEGY DSL

A typed, versioned domain language:

```text
strategy "EventFlowMomentum" version 1 {
  universe = us_equities
  when:
    relative_volume > 2.0
    and options.call_put_premium_ratio > 2.5
    and iv_rank < 45
    and days_to_earnings between 10 and 30
    and regime.volatility != "crisis"
  enter:
    structure = call_debit_spread
    dte between 30 and 60
    long_delta between 0.55 and 0.70
  risk:
    max_premium_at_risk = 0.50% of portfolio
    max_sector_exposure = 15%
  exit:
    profit_target = 50% of max_profit
    stop = 40% of debit
    max_holding_days = 15
}
```

The DSL MUST compile to a typed intermediate representation.

PYTHON

Use a sandboxed research service for exploratory analysis. Python MUST not become the trusted execution core.

Recommended libraries:

- Polars.
- PyArrow.
- NumPy.
- SciPy.
- statsmodels.
- scikit-learn.
- Jupyter-compatible environment where appropriate.

RUST SDK

For production-grade strategies:

- Typed events.
- Deterministic clock.
- Explicit state.
- Bounded memory.

- Reproducible random sources.
- No direct network access.
- Capability-scoped data access.

11.2 Backtesting Engine

The engine MUST model:

- Corporate actions.
- Delistings.
- Survivorship.
- Symbol changes.
- Trading calendars.
- Session boundaries.
- Timezones.
- Bid/ask where available.
- Spread.
- Slippage.
- Commission.
- Market impact approximation.
- Liquidity constraints.
- Partial fills.
- Option expiration.
- Assignment/exercise assumptions.
- Missing or stale data.
- Event publication time.
- Filing availability time.
- Data revisions.

11.3 Validation Modes

- In-sample.
- Holdout.
- Rolling out-of-sample.
- Walk-forward.
- Anchored walk-forward.
- Purged cross-validation for overlapping labels.
- Monte Carlo resampling.
- Parameter perturbation.
- Regime split.
- Sector split.
- Timeframe split.
- Reality-check/multiple-hypothesis controls.

11.4 Metrics

- Total and annualized return.
- Win rate.
- Expectancy.
- Profit factor.
- Max drawdown.
- Ulcer index.
- Sharpe and Sortino, with assumptions.
- Calmar.
- Tail loss.
- CVaR/expected shortfall.
- Exposure.
- Turnover.

- Capacity approximation.
- Maximum favorable/adverse excursion.
- Time under water.
- Slippage sensitivity.
- Regime-specific performance.
- Confidence intervals.

11.5 Monte Carlo Lab

Simulation modes:

22. Historical bootstrap.
23. Block bootstrap.
24. Parametric return distribution.
25. Geometric Brownian motion.
26. Jump diffusion.
27. Stochastic volatility.
28. Regime switching.
29. Correlated multi-asset simulation.
30. Volatility-surface-aware option simulation.
31. Trade-sequence reshuffling.

Outputs:

- Terminal-value distribution.
- Probability of profit.
- Probability of ruin.
- Drawdown distribution.
- Expected shortfall.
- Strike-touch probability.
- Path-dependent stop/target outcomes.
- Time-to-target distribution.
- Volatility shock sensitivity.
- Theta decay scenarios.
- Portfolio breach probability.

Every simulation MUST record its assumptions and seed.

12. Predictive Intelligence Architecture

12.1 Forecast Types

- Direction probability.
- Return distribution.
- Volatility forecast.
- Range forecast.
- Event reaction distribution.
- Liquidity forecast.
- Regime probability.
- Anomaly score.
- Opportunity ranking.
- Risk of thesis invalidation.

12.2 Model Hierarchy

Start simple and earn complexity:

32. Baseline historical frequencies.
33. Linear/logistic models.
34. Tree ensembles.
35. Time-series models.
36. Calibrated gradient boosting.
37. Deep temporal models only where they outperform robust baselines.
38. Regime ensembles.
39. Bayesian models where uncertainty is central.

12.3 Feature Families

- Price and return.
- Volume and liquidity.
- Volatility.
- Options flow and surface.
- Market breadth.
- Sector/industry state.
- Institutional activity.
- COT positioning.
- Filing events.
- Macro regime.
- Crypto category and dominance.
- Cross-asset correlation.
- Sentiment and news where licensed.
- User strategy context.

12.4 Explainability

For each prediction:

- Top positive contributors.
- Top negative contributors.
- Feature freshness.
- Missing features.
- Similar examples.

- Calibration curve.
- Training window.
- Model version.
- Known limitations.
- Data drift status.

12.5 Model Governance

Models MUST have:

- Owner.
- Intended use.
- Prohibited use.
- Dataset lineage.
- Training code version.
- Validation report.
- Bias/leakage review.
- Calibration report.
- Promotion approval.
- Drift thresholds.
- Rollback.
- Retirement plan.

12.6 Avoiding False Precision

The UI SHOULD prefer:

- “Estimated 58–66% probability range”over
- “62.37% chance.”

Confidence must widen when data is sparse, stale, out-of-domain, or contradictory.

13. Options Intelligence Architecture

13.1 Chain Explorer

Capabilities:

- Expiration matrix.
- Strike ladder.
- Calls and puts.
- Bid/ask.
- Last.
- Volume.
- Open interest.
- Implied volatility.
- Greeks.
- Intrinsic/extrinsic value.
- Break-even.
- Probability estimates.
- Liquidity score.
- Spread width.
- Event markers.
- Historical IV context.

13.2 Flow Classification

Each observed trade or aggregate MUST separate facts from inference.

Facts:

- Timestamp.
- Contract.
- Size.
- Price.
- Bid/ask context.
- Premium.
- Venue if available.
- Volume and open interest.
- Underlying state.

Inferences:

- Buy/sell side likelihood.
- Opening/closing likelihood.
- Spread membership likelihood.
- Directional/hedging likelihood.
- Urgency.
- Repeat accumulation.
- Confidence.

13.3 Flow Clustering

Cluster trades by:

- Underlying.
- Actor-like timing pattern.
- Expiration.
- Strike.

- Side.
- Premium.
- Venue.
- Repeated sequence.
- Multi-leg relationship.

The UI SHOULD present both individual prints and inferred parent activity.

13.4 Volatility Lab

- IV rank.
- IV percentile.
- Realized volatility.
- Term structure.
- Skew.
- Smile.
- Event IV.
- Forward volatility.
- Volatility cone.
- Surface interpolation.
- Surface change.
- Relative-value comparisons.

13.5 Strategy Constructor

Inputs:

- Thesis direction.
- Expected move.
- Time horizon.
- Volatility view.
- Risk budget.
- Probability preference.
- Liquidity threshold.
- Event exposure.
- Account constraints.

Candidate structures are ranked by:

- Expected value.
- Probability of profit.
- Maximum loss.
- Capital efficiency.
- Liquidity.
- Theta.
- Vega.
- Gamma.
- Assignment complexity.
- Tail risk.
- Scenario stability.

13.6 Trade Quality Score

Suggested dimensions:

- Evidence confluence.
- Flow quality.
- Catalyst quality.
- Volatility value.
- Technical structure.

- Institutional alignment.
- Liquidity.
- Execution cost.
- Risk/reward.
- Historical validation.
- Model confidence.
- Data quality.
- Portfolio fit.

The final score MUST not hide dimension-level disagreement.

13.7 Dealer Exposure

Gamma and related dealer-exposure estimates SHOULD be clearly labeled as estimates because open-interest ownership and dealer positioning are not directly observable in full.

14. Institutional and Regulatory Intelligence

14.1 13F Intelligence

Functions:

- Filing calendar.
- Manager profiles.
- Current holdings.
- New positions.
- Increases.
- Decreases.
- Exits.
- Concentration.
- Sector changes.
- Manager consensus.
- Position novelty.
- Historical manager behavior.

Limitations MUST be explicit:

- Reporting delay.
- Quarter-end snapshot.
- Long holdings emphasis.
- Incomplete short/derivative picture.
- Unknown intra-quarter transaction timing.

14.2 Ownership Intelligence

13D/13G features:

- Ownership threshold events.
- Amendment history.
- Activist indicators.
- Holder relationships.
- Change magnitude.
- Related news and filings.

14.3 Insider Intelligence

Forms 3/4/5:

- Open-market buys/sells.
- Grants.
- Option exercises.
- Tax-related dispositions.
- 10b5-1 indicators where reported.
- Cluster buying.
- Role and ownership context.
- Historical insider behavior.

The system MUST not treat all insider transactions equally.

14.4 Material Event Intelligence

8-K extraction:

- Earnings and guidance.
- Leadership changes.
- Material agreements.
- Debt events.
- Acquisitions.
- Cybersecurity incidents.
- Restatements.
- Bankruptcy.
- Auditor changes.

14.5 Institutional Relationship Graph

Graph nodes:

- Institution.
- Fund.
- Manager.
- Company.
- Executive.
- Filing.
- Security.
- Sector.
- Event.

Edges MUST retain source, effective date, confidence, and validity interval.

15. Crypto Intelligence

15.1 Crypto Market Command

- Total market capitalization.
- Volume.
- BTC/ETH dominance.
- Stablecoin context where available.
- Category performance.
- Trending assets.
- Largest movers.
- Liquidity warnings.
- Exchange conditions.
- Market regime.

15.2 Asset Profile

- Canonical token identity.
- Chain and contract addresses.
- Market data.
- Supply metrics.
- Market capitalization and fully diluted valuation.
- Historical chart.
- Categories.
- Exchange listings.
- Community/developer metadata where available.
- Related assets.
- Risk flags.
- Data freshness.

15.3 Category and Narrative Explorer

- Category constituents.
- Weighted performance.
- Dispersion.
- Volume.
- Leadership rotation.
- Correlation.
- New entrants.
- Historical cycles.

15.4 Exchange Explorer

- Exchange metadata.
- Trust/quality information supplied by provider.
- Pair availability.
- Volume.
- Geographic or access notes where known.
- User-connected account adapters in later phases.

15.5 Exchange-Native Extensions

CoinGecko is the aggregation foundation, but optional adapters SHOULD provide:

- Order books.
- Trades.
- Funding rates.
- Open interest.
- Liquidations.
- Private balances.
- Orders.
- Execution.

Potential adapters:

- Coinbase.
- Kraken.
- Binance where legally and geographically appropriate.
- Hyperliquid.
- Bybit where legally appropriate.

These are separate security and compliance boundaries.

15.6 Crypto Risk Engine

- Liquidity concentration.
- Exchange concentration.
- Volatility.
- Drawdown.
- Correlation.
- Token age.
- Supply concentration where data exists.
- Contract/chain mapping warnings.
- Stablecoin depeg scenarios.
- Bridge/protocol exposure through later providers.
- Venue outage scenarios.

16. AI and Agentic Intelligence

16.1 AI Analyst

Supported questions:

- Why did this asset move?
- What upcoming events matter?
- What changed in the options surface?
- Which institutional filings are significant?
- Find similar historical conditions.
- Construct and compare option structures.
- Explain this strategy's failure.
- Summarize portfolio risks.
- Generate a research plan.
- Translate natural language into a scan or strategy draft.

16.2 Tool Architecture

The AI MUST use a controlled tool layer:

- Read-only market query.
- Filing query.
- COT query.
- Backtest request.
- Simulation request.
- Chart snapshot.
- Portfolio analysis.
- Journal search.
- Draft alert.
- Draft strategy.
- Draft order.

Live order submission MUST not be directly available to a general conversational model. It requires a deterministic execution workflow and explicit user approval.

16.3 MCP Integration

MCP servers MUST be:

- Explicitly installed.
- Signed where possible.
- Version pinned.
- Capability scoped.
- Network restricted.
- Audited.
- Disableable.
- Assigned a trust level.
- Prevented from reading unrelated local files or secrets.

A remote MCP server SHOULD be treated as an external dependency, not trusted local code.

16.4 Retrieval and Knowledge

The AI retrieval layer may index:

- User notes.
- Strategy definitions.
- Backtest reports.
- Journal entries.
- Filings.
- Provider documentation.
- Internal help.
- Model cards.
- Runbooks.

Sensitive indices MUST be encrypted, tenant-scoped, and covered by retention policy.

16.5 AI Output Contract

Every market answer SHOULD provide:

- Direct answer.
- Evidence.
- Source timestamps.
- Uncertainty.
- Contradicting evidence.
- Suggested next verification.
- Risk note.
- Tool calls used.
- Model/version metadata when appropriate.

17. Portfolio, Risk, Journal, and Execution

17.1 Portfolio Model

Support:

- Multiple accounts.
- Manual portfolios.
- Paper portfolios.
- Broker-linked portfolios.
- Tax lots where data permits.
- Cash.
- Equities.
- Options.
- Crypto.
- Strategies and sleeves.

17.2 Risk Measures

- Gross and net exposure.
- Delta-adjusted exposure.
- Greeks.
- Sector/industry concentration.
- Asset and venue concentration.
- Correlation clusters.
- Beta.
- Volatility.
- Drawdown.
- Scenario P/L.
- Event exposure.
- Liquidity.
- Margin approximation where appropriate.
- Tail-risk indicators.

VaR MUST not be the sole risk representation.

17.3 Pre-Trade Checks

- Max position.
- Max premium at risk.
- Max account loss.
- Sector concentration.
- Correlation.
- Event proximity.
- Spread width.
- Volume/open interest.
- Account permissions.
- Market status.
- Stale data.
- Duplicate order.
- Strategy conflict.
- Daily loss limit.

17.4 Journal

Automatic fields:

- Order and fill.
- Chart snapshot.
- Market regime.
- Options state.
- Event state.
- Strategy version.
- Risk plan.
- Outcome.

Manual fields:

- Thesis.
- Confidence.
- Emotional state.
- Mistake tags.
- What would invalidate.
- Review notes.

AI analysis SHOULD identify patterns but MUST not make unsupported psychological diagnoses.

17.5 Execution Roadmap

Execution SHOULD be deferred until research and paper trading are reliable.

Stages:

40. Draft orders.
41. Paper execution.
42. Read-only broker account sync.
43. User-approved live orders.
44. Rule-based automation with approval.
45. Limited unattended automation with strict controls.

Every order workflow MUST use idempotency keys and reconcile broker state after uncertain outcomes.

MYTHOS SYSTEMS / PRISMATIK

05

PLATFORM & SYSTEM ARCHITECTURE

Rust-centered system design, Tauri/Svelte experience, storage, streaming, domain models, and extensibility.

CORNERSTONE EXECUTIVE ARCHITECTURE REPORT | 2026

18. System Architecture

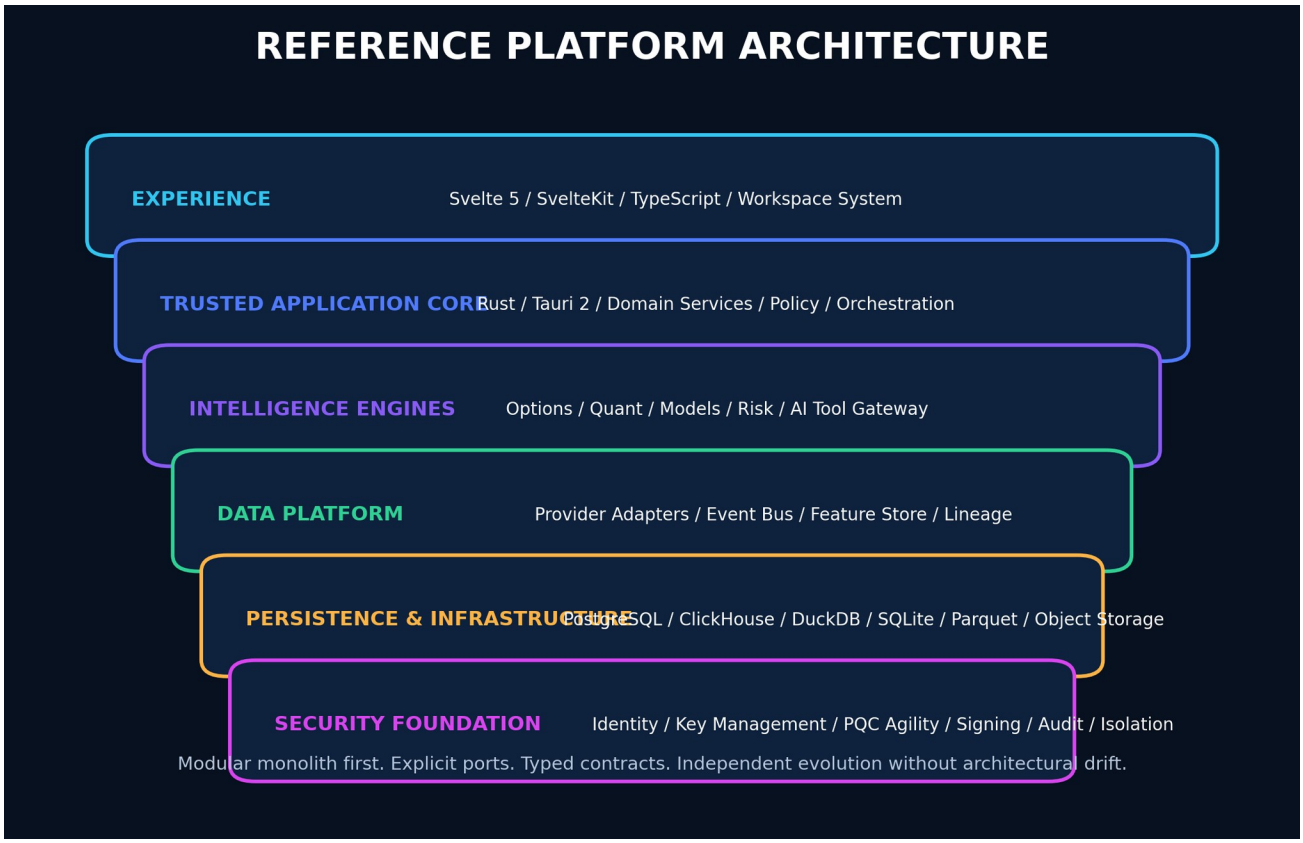

Figure 6 - Reference platform architecture for the desktop, cloud, and on-premises product.

18.1 Recommended Architecture Style

Begin as a modular monolith with isolated crates and process boundaries where risk or scale justifies them.

Primary processes:

46. **Desktop UI process** — Tauri WebView running Svelte.
47. **Trusted Rust core** — Tauri commands, local orchestration, security, domain services.
48. **Ingestion workers** — provider adapters and schedulers.
49. **Quant worker** — backtests and simulation jobs.
50. **GPU renderer/compute module** — wgpu.
51. **Optional local AI runtime connector.**
52. **Optional cloud control plane.**

18.2 High-Level Components

```text
┌──────────────────────────────────────────────────────────────┐
│ PRISMATIK Desktop / Web                                      │
│ Svelte 5 + TypeScript + SvelteKit + Design System            │
└──────────────────────────────┬───────────────────────────────┘
                               │ Typed IPC / HTTPS / WebSocket
┌──────────────────────────────▼───────────────────────────────┐
│ Rust Application Core                                        │
│ Identity | Workspaces | Market | Options | Crypto | Events | Risk │
│ Strategies | Simulation | Portfolio | Journal | AI Tool Gateway │
└──────────────────────────────┬───────────────────────────────┘
                               │
        ┌──────────────────────┼──────────────────────┐
        ▼                      ▼                      ▼
┌──────────────┐      ┌──────────────┐      ┌──────────────┐
│ Data Gateway │      │ Quant Engine │      │ Security Core │
│ Adapters     │      │ Backtest/MC  │      │ Keys/Policy   │
└──────┬───────┘      └──────┬───────┘      └──────┬────────┘
       │                     │                     │
┌──────▼─────────────────────▼─────────────────────▼──────────┐
│ PostgreSQL | ClickHouse | DuckDB | SQLite | Arrow/Parquet | Cache │
└──────────────────────────────────────────────────────────────┘
```

18.3 Rust Responsibilities

- Domain models.
- Provider clients.
- Authentication and authorization.
- Secret management.
- Persistence.
- Scheduling.
- Event processing.
- Quantitative engine.
- Risk engine.
- Order orchestration.
- File import/export.
- Cryptography.
- Audit.
- Native notifications.
- Auto-update verification.
- Plugin validation.
- System health.

18.4 TypeScript/Svelte Responsibilities

- Presentation.
- Interaction.
- Layout.
- Accessibility.
- Local ephemeral UI state.
- Typed API consumption.
- Visualization orchestration.
- Form validation in addition to backend validation.
- Offline UI.
- Command palette.
- Workspace composition.

TypeScript MUST not own authoritative trading, authorization, risk, or cryptographic decisions.

18.5 wgpu Responsibilities

Use wgpu selectively for:

- Large candlestick datasets.

- Order-flow heatmaps.
- Volatility surfaces.
- Monte Carlo path visualization.
- Correlation matrices.
- Institutional graphs.
- Dense scatter/point clouds.
- Animated market maps.
- Selected parallel numerical kernels after benchmarking.

Maintain CPU and Canvas/SVG fallbacks.

19. Repository and Workspace Architecture

```text
/
├─ apps/
│  ├─ desktop/                # Tauri 2 + SvelteKit
│  ├─ web/                    # Optional browser companion
│  └─ admin/                  # Enterprise operations console
├─ crates/
│  ├─ prismatik-domain/
│  ├─ prismatik-application/
│  ├─ prismatik-market-data/
│  ├─ prismatik-options/
│  ├─ prismatik-crypto/
│  ├─ prismatik-filings/
│  ├─ prismatik-cot/
│  ├─ prismatik-events/
│  ├─ prismatik-strategy/
│  ├─ prismatik-backtest/
│  ├─ prismatik-simulation/
│  ├─ prismatik-risk/
│  ├─ prismatik-portfolio/
│  ├─ prismatik-journal/
│  ├─ prismatik-ai-tools/
│  ├─ prismatik-security/
│  ├─ prismatik-storage/
│  ├─ prismatik-observability/
│  ├─ prismatik-renderer/
│  └─ prismatik-cli/
├─ packages/
│  ├─ ui/
│  ├─ design-tokens/
│  ├─ api-client/
│  ├─ schemas/
│  └─ chart-contracts/
├─ services/
│  ├─ ingest/
│  ├─ quant-worker/
│  ├─ notification/
│  └─ cloud-api/
├─ schemas/
│  ├─ openapi/
│  ├─ json-schema/
│  ├─ events/
│  └─ database/
├─ infra/
│  ├─ containers/
│  ├─ kubernetes/
│  ├─ terraform/
│  └─ on-prem/
├─ docs/
│  ├─ ENGINEERING_STANDARDS.md
│  ├─ architecture.md
│  ├─ security.md
│  ├─ api.md
│  ├─ data-model.md
│  ├─ observability.md
│  ├─ testing.md
│  ├─ threat-model/
│  ├─ runbooks/
│  ├─ adr/
│  ├─ diagrams/
│  ├─ model-cards/
│  └─ workbook/
├─ tests/
├─ scripts/
├─ .github/workflows/
├─ README.md
├─ CONTRIBUTING.md
├─ SECURITY.md
├─ CHANGELOG.md
└─ Cargo.toml
```

20. Core Domain Model

20.1 Key Aggregates

- Asset.
- Venue.
- MarketSnapshot.
- BarSeries.
- OptionContract.
- OptionChain.
- FlowObservation.
- Event.
- Filing.
- Institution.
- PositionDisclosure.
- Strategy.
- BacktestRun.
- SimulationRun.
- Opportunity.
- Portfolio.
- Position.
- OrderIntent.
- Order.
- Fill.
- RiskPolicy.
- JournalEntry.
- ProviderAccount.
- DataEntitlement.
- Model.
- ModelPrediction.
- Workspace.
- AlertRule.
- Automation.

20.2 Opportunity Aggregate

```rust
pub struct Opportunity {
    pub id: OpportunityId,
    pub asset_id: AssetId,
    pub thesis: Thesis,
    pub horizon: TimeHorizon,
    pub evidence: Vec<EvidenceRef>,
    pub contradictions: Vec<EvidenceRef>,
    pub score: OpportunityScore,
    pub expected_value: Option<DistributionSummary>,
    pub liquidity: LiquidityAssessment,
    pub risk: RiskAssessment,
    pub freshness: FreshnessSummary,
    pub provenance: ProvenanceBundle,
    pub model_versions: Vec<ModelVersionId>,
    pub created_at: time::OffsetDateTime,
    pub expires_at: Option<time::OffsetDateTime>,
}
```

20.3 Immutable Evidence

Evidence SHOULD be immutable and content-addressed. Corrections create superseding records rather than destructive edits.

21. Storage and Data Lifecycle

21.1 Storage Roles

POSTGRESQL

- Users.
- Organizations.
- Accounts.
- Configuration.
- Strategies.
- Portfolios.
- Orders.
- Journals.
- Audit metadata.
- Entitlements.
- Model registry.

CLICKHOUSE

- High-volume time series.
- Options observations.
- Market events.
- Flow records.
- Analytics aggregates.
- Telemetry.

DUCKDB

- Local analytics.
- Offline research.
- Parquet query.
- Portable project bundles.
- Fast embedded aggregations.

SQLITE

- Desktop configuration.
- Cached metadata.
- Offline queue.
- Workspace state.
- Local audit buffer.

ARROW/PARQUET

- Columnar interchange.
- Historical datasets.
- Backtest snapshots.
- Export.
- Reproducible research artifacts.

OBJECT STORAGE

- Filing documents.
- Model artifacts.
- Backtest manifests.
- Simulation outputs.

- Chart snapshots.
- Signed exports.
- Encrypted backups.

21.2 Retention Classes

- Ephemeral cache.
- Operational data.
- Market history.
- User content.
- Regulatory source documents.
- Audit.
- Security logs.
- Model artifacts.
- Backups.

Each class MUST define:

- Owner.
- Retention.
- Encryption.
- Access.
- Export.
- Deletion.
- Legal hold.
- Backup.
- Restore objective.

21.3 Local Encryption

Local databases SHOULD use application-layer envelope encryption for sensitive columns in addition to platform disk encryption.

22. Real-Time Streaming and Event Architecture

22.1 Event Bus

For local/modular deployment:

- Tokio channels for intra-process messaging.
- Durable outbox for state changes.
- SQLite/PostgreSQL job queue where sufficient.

For scaled deployment:

- NATS JetStream SHOULD be evaluated first.
- Kafka/Redpanda MAY be used for large sustained event volume and replay requirements.

22.2 Event Envelope

```json
{
  "event_id": "01J...",
  "event_type": "MarketSnapshotUpdated",
  "schema_version": 1,
  "produced_at": "2026-07-13T15:00:00Z",
  "producer": "coingecko-adapter",
  "correlation_id": "01J...",
  "tenant_id": null,
  "data_classification": "public-market-data",
  "payload": {}
}
```

22.3 Reliability

- Transactional outbox.
- Idempotent consumers.
- Dead-letter queues.
- Replay controls.
- Schema compatibility.
- Bounded queues.
- Backpressure.
- Per-provider concurrency limits.
- Poison-message quarantine.

MYTHOS SYSTEMS / PRISMATIK

06

SECURITY, PQC & ENTERPRISE OPERATIONS

Zero-trust controls, post-quantum agility, in-memory protection, on-prem deployment, observability, and release security.

CORNERSTONE EXECUTIVE ARCHITECTURE REPORT | 2026

23. Security Architecture

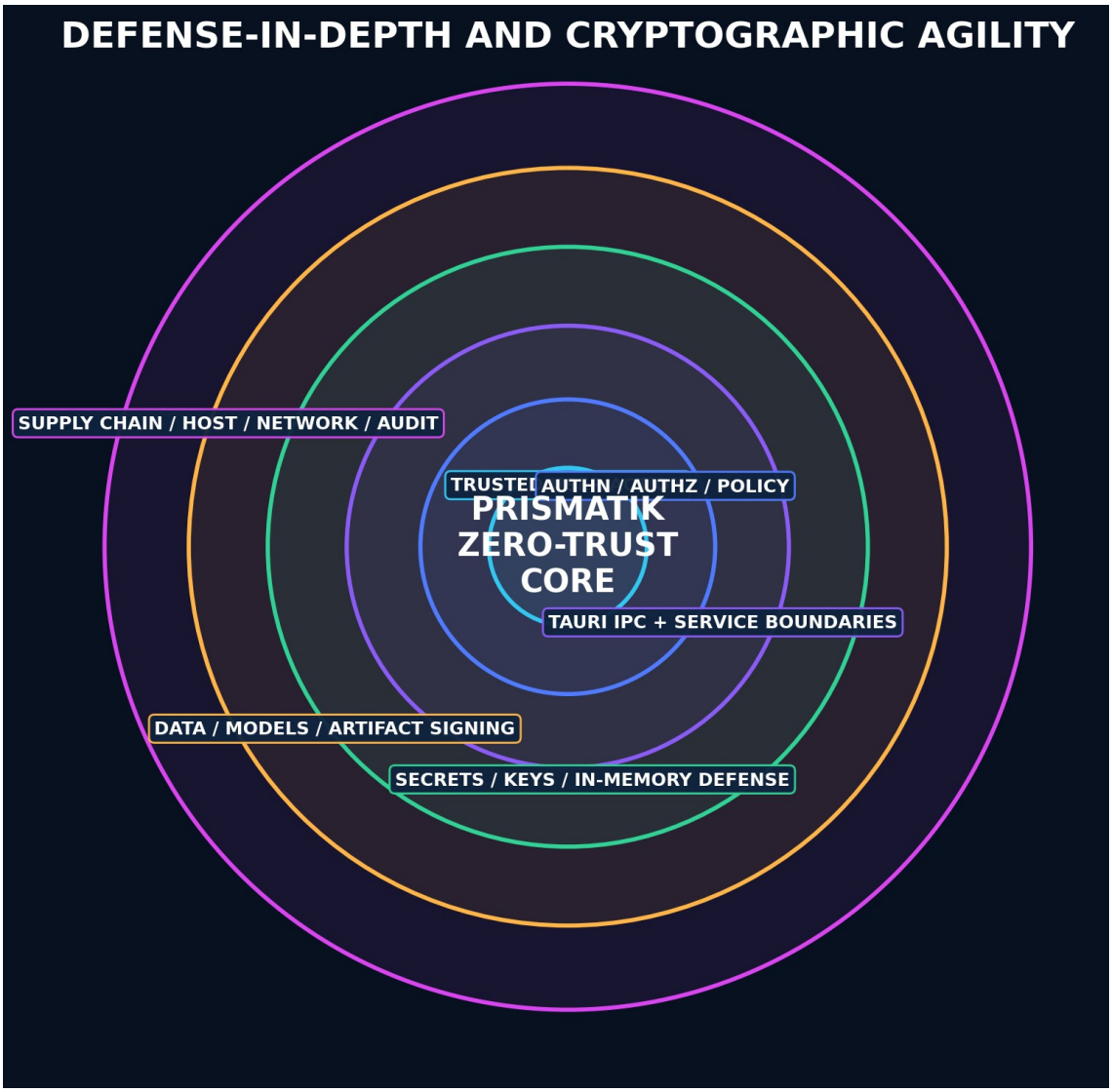

Figure 7 - Defense-in-depth security model protecting user action, keys, services, data, models, and the supply chain.

23.1 Threat Model Assets

- Broker credentials.
- Exchange API keys.
- User identity.
- Portfolio data.
- Trade history.
- Strategy IP.
- AI context.
- Provider API keys.
- Cryptographic keys.
- Update trust roots.
- Plugin/MCP permissions.
- Audit logs.
- Model artifacts.

23.2 Primary Threats

- Credential theft.
- Unauthorized trading.
- Malicious plugin or MCP server.
- Supply-chain compromise.
- IPC abuse.
- WebView injection.
- XSS and content injection from filings/news.
- SSRF through provider connectors.
- Replay and duplicate orders.
- Data poisoning.
- Model manipulation.
- Stale-data execution.
- Privilege escalation.
- Memory scraping.
- Local database theft.
- Update hijacking.
- Insider misuse.
- Denial of service.
- Dependency compromise.

23.3 Authentication

Recommended:

- OIDC/OAuth 2.1 for cloud identities.
- Passkeys/WebAuthn.
- MFA.
- Device-bound sessions.
- Short-lived access tokens.
- Rotating refresh tokens.
- Step-up authentication for:
  - Adding broker credentials.
  - Enabling live execution.
  - Changing risk limits.
  - Exporting sensitive data.
  - Installing plugins.
  - Disabling audit/security controls.

23.4 Authorization

- Organization.
- Workspace.
- Account.
- Strategy.
- Dataset.
- Execution permission.
- Administration.
- Audit access.

Use RBAC plus ABAC for account, environment, device posture, and risk context.

23.5 Tauri Security

- Minimal capabilities per window/webview.
- Strict Content Security Policy.
- No arbitrary shell access.

- Validate all IPC payloads in Rust.
- Explicit command allowlist.
- Separate high-risk administrative window capability.
- Signed updater.
- Restricted asset/file protocols.
- No secrets in frontend state.
- Disable unnecessary plugins.
- Origin checks.
- Secure deep-link handling.

23.6 API Security

- TLS 1.3.
- mTLS for service-to-service where appropriate.
- OAuth scopes.
- Request signing for high-risk integrations where supported.
- Rate limits.
- Idempotency keys.
- Replay windows.
- Nonces where appropriate.
- Input schemas.
- Output encoding.
- Safe error envelopes.
- Audit records.
- Correlation IDs.

23.7 Secret Management

Desktop:

- Windows Credential Manager/DPAPI.
- macOS Keychain/Secure Enclave-backed storage where supported.
- Linux Secret Service/keyring.
- TPM-backed wrapping where feasible.

Server/on-prem:

- HashiCorp Vault or cloud secret manager.
- HSM/KMS-backed master keys.
- Dynamic credentials where possible.
- Short-lived broker/exchange tokens when supported.

Secrets MUST never enter browser JavaScript bundles, logs, crash reports, analytics, or workbooks.

23.8 In-Memory Security

- Use `secrecy` types for secret material.
- Zeroize buffers with `zeroize`.
- Minimize copies.
- Avoid long-lived `String` values for secrets.
- Disable or restrict core dumps.
- Lock sensitive pages where practical and justified.
- Protect swap/hibernation through OS controls and disk encryption.
- Avoid secrets in exception/context chains.
- Constant-time comparisons for secret values.
- Separate execution credentials into a dedicated process in high-security deployments.
- Use short token lifetimes.
- Clear clipboard after timed secret copy.
- Redact screenshots and diagnostics.

23.9 Database Security

- Encryption in transit.
- Encryption at rest.
- Row-level security for multi-tenant deployments.
- Tenant keys where required.
- Least-privilege service accounts.
- Query allowlists for agent tools.
- Migration signing/checksums.
- Point-in-time recovery.
- Backup encryption.
- Restore drills.

23.10 Data Poisoning Defenses

- Provider provenance.
- Cross-source comparison.
- Outlier detection.
- Signed dataset manifests.
- Immutable raw snapshots.
- Parser versioning.
- Quarantine suspicious data.
- Human review for model-training datasets.
- Feature-level quality flags.
- No silent correction.

24. Post-Quantum and Cryptographic Agility Program

24.1 Scope

Post-quantum readiness applies primarily to:

- Key establishment.
- Public-key encryption/wrapping.
- Digital signatures.
- Long-lived data confidentiality.
- Software/update signing.
- Service identity.
- Backup/export protection.
- Plugin and model artifact verification.

It does not replace modern symmetric encryption, access control, memory protection, or secure operations.

24.2 Approved Direction

Use NIST-standardized algorithms through maintained, reviewed implementations:

- **ML-KEM** for key establishment.
- **ML-DSA** for general digital signatures.
- **SLH-DSA** where a stateless hash-based signature is appropriate.

24.3 Migration Strategy

NEAR TERM

- Maintain a crypto bill of materials.
- Use TLS 1.3.
- Use X25519/P-256 ECDHE with AES-256-GCM or ChaCha20-Poly1305 when PQ hybrid modes are unavailable.
- Use Ed25519/ECDSA for signatures where required today.
- Version all key and signature formats.
- Support multiple algorithms per trust policy.
- Design dual-signature manifests.
- Protect long-lived data keys through envelope encryption.

HYBRID PHASE

Where runtime and ecosystem support are proven:

- Hybrid classical + ML-KEM key establishment.
- Dual-sign release and plugin manifests using classical + ML-DSA.
- PQC-capable internal tunnels or service mesh where supported.
- Interoperability and downgrade testing.

MATURE PHASE

- PQC-primary profiles.
- Classical compatibility fallback only by explicit policy.
- PQ-aware certificate and identity lifecycle.
- Automated algorithm deprecation.

24.4 Crypto Bill of Materials

Track:

- Component.
- Protocol.
- Algorithm.
- Parameters.
- Library.
- Key owner.
- Storage.
- Rotation.
- Expiration.
- Data lifetime.
- Quantum risk.
- Migration trigger.
- Fallback.
- Verification evidence.

24.5 Artifact Signing

Sign:

- Desktop releases.
- Installers.
- Update manifests.
- Containers.
- SBOMs.
- Plugin packages.
- MCP bundles.
- Model artifacts.
- Strategy packages.
- Dataset manifests.
- Migration bundles.

Design manifests for dual signatures.

24.6 Local Vault

A local encrypted vault MAY use:

- Random data-encryption keys.
- AES-256-GCM or XChaCha20-Poly1305.
- Per-record or per-domain keys.
- OS/hardware-backed key-encryption keys.
- Optional recovery key protected by a user-held secret.
- Versioned envelope format.
- Rotation and rewrap without re-encrypting all data where possible.

Do not invent a custom cipher or protocol.

24.7 PQC Exception Process

Every boundary without practical PQC support MUST document:

- Missing capability.
- Reason.
- Classical fallback.
- Data lifetime.
- Harvest-now/decrypt-later risk.

- Owner.
- Review date.
- Migration trigger.

25. On-Premises, Air-Gapped, and Enterprise Deployment

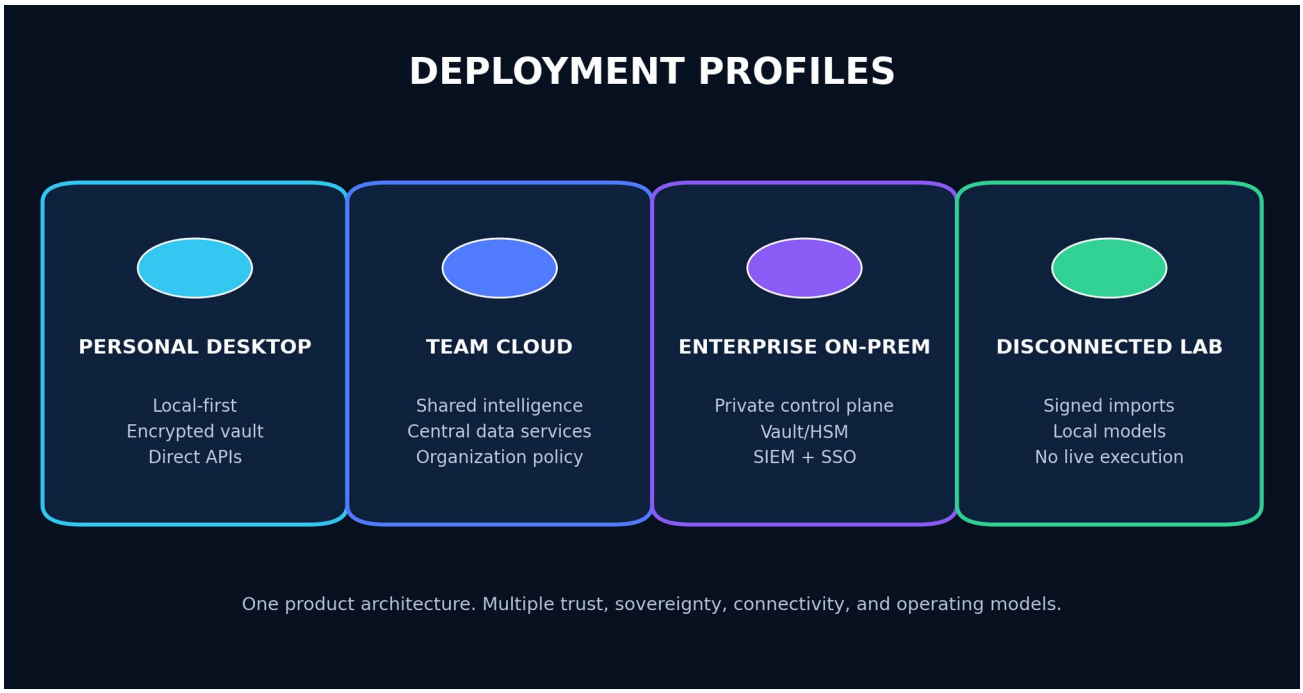

Figure 8 - Deployment profiles sharing one core architecture with different trust and connectivity models.

25.1 Deployment Profiles

PERSONAL DESKTOP

- Local database.
- Local encrypted vault.
- Direct provider APIs.
- Optional cloud sync.
- Local or remote AI.

TEAM CLOUD

- Central API.
- Shared datasets.
- PostgreSQL/ClickHouse.
- Organization policy.
- Central observability.
- Shared research and audit.

ENTERPRISE ON-PREM

- Kubernetes or hardened VM deployment.
- Private registry.
- Vault/HSM.
- Enterprise IdP.
- Proxy support.
- Internal CA.
- SIEM export.

- Restricted egress.
- Data residency controls.

DISCONNECTED RESEARCH ENVIRONMENT

- Signed dataset imports.
- Offline research.
- No live execution.
- Signed export bundles.
- Local AI models.
- Local license/entitlement cache with defined expiration.

25.2 On-Prem Security

- Network segmentation.
- Private service endpoints.
- Management-plane isolation.
- Bastion or zero-trust access.
- mTLS.
- HSM/Vault.
- Immutable audit export.
- SIEM integration.
- EDR compatibility.
- Host hardening.
- CIS-aligned configuration.
- Signed container admission.
- Egress allowlists.
- Backup isolation.
- Disaster recovery exercises.

25.3 Tenant Isolation

Options:

- Shared database with RLS for standard SaaS.
- Schema-per-tenant for stronger logical separation.
- Database-per-tenant for regulated or large enterprise.
- Dedicated deployment for highest assurance.

26. Observability, Reliability, and Operations

26.1 OpenTelemetry

Use OpenTelemetry-compatible:

- Traces.
- Metrics.
- Logs.
- Context propagation.

26.2 Required Metrics

- Provider request count/latency/errors.
- Rate-limit remaining.
- Cache hit rate.
- Freshness lag.
- Stream disconnects.
- Queue depth.
- Job duration.
- Backtest throughput.
- Simulation throughput.
- Model inference latency.
- UI frame time.
- GPU fallback rate.
- Database latency.
- Order reconciliation latency.
- Alert delivery success.
- Security events.

26.3 SLO Examples

- Cached dashboard availability.
- Critical provider freshness.
- Alert delivery.
- Order-state reconciliation.
- Backtest completion.
- Desktop crash-free sessions.
- Update success.

26.4 Health Model

- Liveness.
- Readiness.
- Provider dependency status.
- Database status.
- Clock skew.
- Entitlement status.
- Crypto key status.
- Model status.
- Queue status.
- Version/build.
- Data freshness.

26.5 Runbooks

- Provider outage.
- Rate-limit exhaustion.
- Schema change.
- Stale data.
- Broker timeout.
- Unknown order state.
- Database failover.
- Restore.
- Key rotation.
- Certificate expiration.
- Plugin compromise.
- Model rollback.
- Emergency execution disablement.
- Corrupted cache.
- GPU device loss.

27. Testing and Quality Engineering

27.1 Test Layers

- Unit.
- Property.
- Integration.
- Contract.
- End-to-end.
- Golden.
- Fuzz.
- Performance.
- Chaos/failure injection.
- Security.
- Accessibility.
- Visual regression.
- Model validation.

27.2 High-Value Property Tests

- Option payoff invariants.
- Portfolio accounting conservation.
- Corporate-action adjustment.
- Order idempotency.
- Event deduplication.
- Timezone boundaries.
- Simulation reproducibility.
- Risk-limit monotonicity.
- Encryption round-trip.
- Parser robustness.
- Provider normalization.

27.3 Provider Contract Tests

- Recorded fixtures.
- Schema validation.
- Pagination.
- Rate-limit behavior.
- Error mapping.
- Missing fields.
- Decimal precision.
- Timestamp interpretation.
- Identifier mapping.
- Sandbox/live parity where available.

27.4 Quant Validation

- Known synthetic datasets.
- Golden strategy results.
- Independent calculation comparisons.
- Bias tests.
- Leakage tests.
- Random-seed reproducibility.

- Numeric tolerance policy.
- Cross-language parity for critical math.

27.5 Security Testing

- SAST.
- Dependency audit.
- Secret scanning.
- DAST for cloud APIs.
- Fuzzing parsers.
- IPC abuse tests.
- CSP tests.
- Plugin sandbox tests.
- Update verification tests.
- Authz matrix tests.
- Replay/idempotency tests.
- Threat-model regression tests.

28. Supply-Chain and Release Security

Required:

- Cargo and JavaScript lockfiles.
- Dependency pinning policy.
- `cargo audit`.
- `cargo deny`.
- npm/pnpm audit strategy.
- License scanning.
- SBOM: CycloneDX or SPDX.
- SLSA-style provenance.
- Reproducible release pipeline where feasible.
- Sigstore/cosign for containers.
- Platform code signing.
- Tauri updater signing.
- Protected release keys.
- Two-person release approval for production.
- Signed plugin registry metadata.
- Dependency allow/deny list.
- Renovation with staged testing.
- No AI-added dependency without justification.

29. Performance Engineering

29.1 Performance Budgets

Define budgets for:

- Cold start.
- Warm start.
- Workspace restore.
- Search.
- Chart pan/zoom.
- Streaming update.
- Scan execution.
- Backtest throughput.
- Simulation latency.
- Memory.
- Disk cache.
- Network usage.
- Battery on laptops.

29.2 Numeric Computing

Recommended:

- Polars for dataframe operations.
- Arrow for zero-copy interchange.
- Rayon for CPU parallelism where appropriate.
- Tokio for asynchronous I/O, not CPU work.
- Dedicated compute pools.
- SIMD through maintained crates where profiling supports it.
- GPU compute only after benchmark validation.

29.3 Decimal Precision

Financial quantities MUST use explicit decimal or fixed-point representations where exact monetary arithmetic is required. Floating point is acceptable for statistical models with documented tolerances, not for authoritative cash accounting.

29.4 UI Performance

- Virtualize large lists.
- Batch streaming updates.
- Avoid global reactive fan-out.
- Use derived state.
- Keep chart data outside DOM.
- Progressive loading.
- Worker threads where available.
- GPU resource reuse.
- Device-loss recovery.

30. Accessibility, Localization, and Responsible UX

- WCAG 2.2 AA target.
- Complete keyboard operation.
- Screen-reader semantics.
- High contrast.
- Reduced motion.
- Color-blind-safe modes.
- No red/green-only meaning.
- Adjustable density.
- Local time and UTC views.
- Locale-aware numbers.
- Explicit currency.
- Plain-language risk disclosures.
- Confirmation for irreversible actions.
- Cooling-off option after large losses.
- Configurable daily loss warnings.
- No celebratory animation for risky trades.

31. API, Plugin, MCP, and SDK Platform

31.1 Public API

- REST/OpenAPI for broad interoperability.
- WebSocket for live updates.
- gRPC for selected internal services.
- Versioned schemas.
- OAuth scopes.
- Idempotency.
- Pagination.
- Correlation IDs.
- Signed webhooks.

31.2 Plugin Types

- Data provider.
- Indicator.
- Strategy.
- Visualization.
- Exporter.
- Notification.
- Broker/exchange.
- AI tool.
- Workspace panel.

31.3 Plugin Security

- Signed package.
- Manifest.
- Declared capabilities.
- Version constraints.
- Network allowlist.
- Filesystem scope.
- CPU/memory budget.
- Revocation.
- Audit.
- User/admin approval.
- Sandboxed execution, preferably WebAssembly/WASI for untrusted extensions.

31.4 SDKs

- Rust.
- Python.
- TypeScript.
- CLI.
- OpenAPI-generated clients.

31.5 MCP Gateway

Instead of letting every MCP server connect directly to the desktop, route approved servers through a gateway that enforces:

- Tool allowlist.
- Schema validation.
- Secret isolation.
- Rate limit.
- Audit.
- Output size limit.
- Content filtering.
- Timeout.
- Network policy.
- Human approval for sensitive tools.

MYTHOS SYSTEMS / PRISMATIK

07

DELIVERY ROADMAP & GOVERNANCE

Phased implementation, team model, acceptance criteria, risk register, technology decisions, and the initial backlog.

CORNERSTONE EXECUTIVE ARCHITECTURE REPORT | 2026

32. Phased Implementation Roadmap

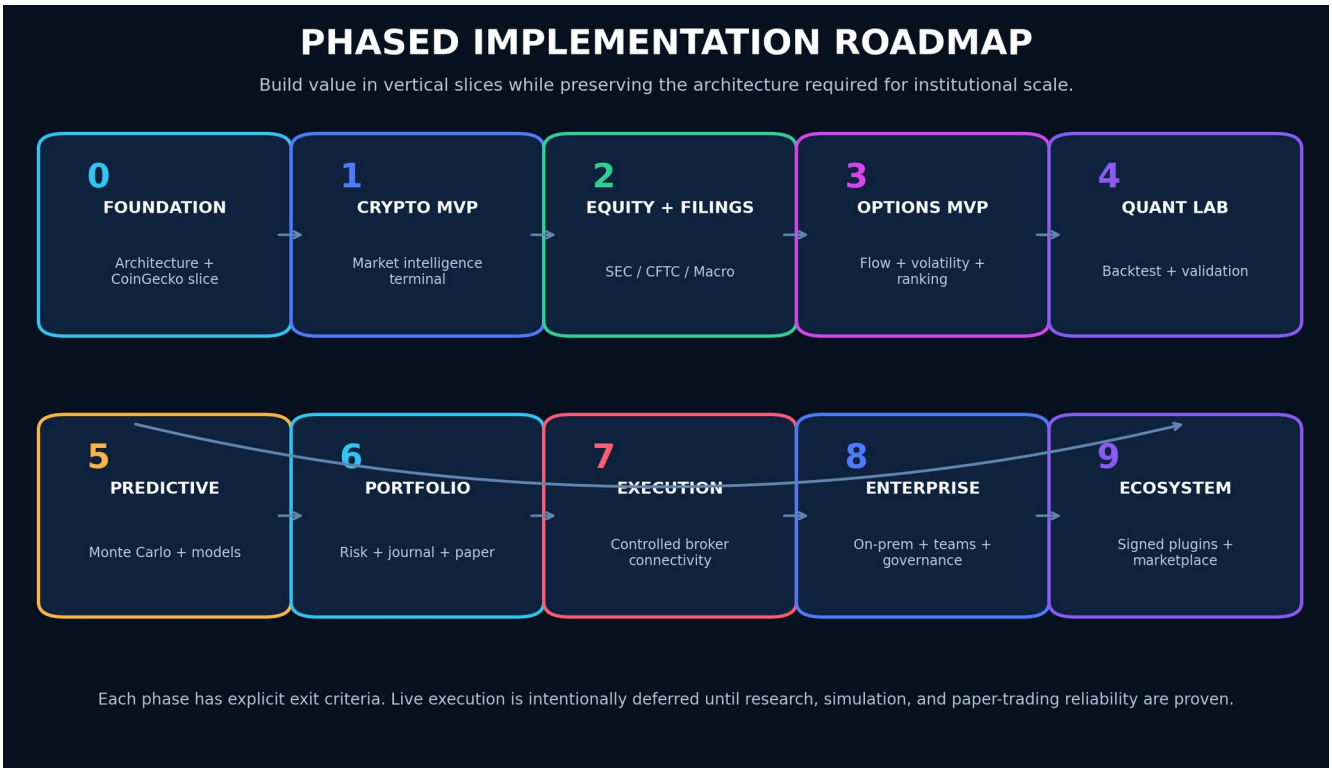

Figure 9 - Phase 0 through Phase 9 implementation roadmap.

Phase 0 — Foundation and Product Proof

Objective: Establish architecture, standards, identity, and a functioning vertical slice.

Deliverables:

- Confirm working product name and namespace policy.
- Repository baseline.
- Engineering standards.
- ADR process.
- Workbook process.
- Threat model.
- Tauri 2 + SvelteKit shell.
- Design tokens and motion system.
- Rust domain/application boundaries.
- Local encrypted configuration.
- CoinGecko demo integration.
- Canonical crypto asset identity.
- Market overview.
- Asset search and detail.
- Basic historical chart.
- Provider health and rate-budget UI.
- OpenTelemetry baseline.
- CI gates.
- Signed development builds.

Exit criteria:

- Desktop app starts reliably on Windows.
- CoinGecko data is cached, validated, and provenance-marked.
- No API key reaches the frontend.

- Core contracts have tests.
- UI meets initial performance and accessibility budgets.

Phase 1 — Crypto Intelligence MVP

Objective: Deliver a polished crypto research terminal using CoinGecko.

Deliverables:

- Global market dashboard.
- Watchlists.
- Asset profiles.
- Categories.
- Trending.
- Exchanges.
- Historical OHLC/market chart.
- Market scanner.
- Alerts.
- Portfolio tracking.
- Correlation and volatility.
- Crypto news where plan permits.
- Local-first workspaces.
- Data-quality inspector.
- Export to CSV/Parquet.
- AI read-only crypto analyst.

Advanced additions:

- Narrative rotation map.
- Category breadth.
- Liquidity-risk scoring.
- Regime classifier.
- Historical analogs.

Exit criteria:

- Product is genuinely useful without equity/options features.
- Rate limits and cache operate under realistic load.
- AI answers show sources, freshness, and uncertainty.
- Offline workspace remains functional with cached data.

Phase 2 — U.S. Equity, Filing, and Macro Foundation

Objective: Add the context needed for institutional and event-driven research.

Deliverables:

- Equity instrument registry.
- SEC EDGAR ingestion.
- Company filings.
- 8-K event extraction.
- 13F holdings.
- 13D/13G.
- Form 4.
- CFTC COT.
- FRED macro.
- Event calendar.
- Filing diff and summary.
- Institutional graph.
- Market regime.
- Equity watchlists and scanner.
- Licensed quote provider evaluation/integration.

Exit criteria:

- Filing records are reproducible and linked to originals.
- COT timing and limitations are clearly shown.
- Event engine supports revisions.
- No model uses data before it was publicly available.

Phase 3 — Options Intelligence MVP

Objective: Establish the primary product wedge.

Deliverables:

- Unusual Whales integration.
- Option chain.
- Flow feed.
- Sweep/block clustering.
- Open interest.
- Volatility lab.
- Implied move.
- Strategy constructor.
- Payoff diagrams.
- Liquidity score.
- Event-aware option ranking.
- Opportunity cards.
- Alert rules.
- Provider entitlement registry.

Exit criteria:

- Every option opportunity is explainable.
- Flow inference is confidence-scored.
- Strategy P/L math passes independent golden tests.
- Delayed/stale data cannot trigger live workflows.

Phase 4 — Quant Research and Backtesting

Objective: Allow users to test ideas credibly.

Deliverables:

- Strategy DSL.
- Visual strategy builder.
- Dataset snapshots.
- Backtest engine.
- Execution simulation.
- Walk-forward validation.
- Parameter analysis.
- Regime testing.
- Reproducibility manifests.
- Python research bridge.
- Rust production strategy SDK.
- Research report generator.

Exit criteria:

- Bias and leakage test suite passes.
- Results reproduce from manifest.
- Corporate actions and event timestamps are handled.
- Baseline performance is competitive for target workloads.

Phase 5 — Monte Carlo and Predictive Intelligence

Objective: Quantify uncertainty and rank opportunities probabilistically.

Deliverables:

- Monte Carlo lab.
- Historical/bootstrap simulation.
- Stochastic models.
- Portfolio simulations.
- Forecast registry.
- Model cards.
- Calibration.
- Drift monitoring.
- Explainability.
- Opportunity fusion.
- Contradiction detection.
- Historical analog engine.

Exit criteria:

- Probabilities are calibrated on held-out data.
- Every model has an owner and rollback.
- Out-of-domain conditions widen uncertainty or suppress output.
- Simulation assumptions are transparent.

Phase 6 — Portfolio, Journal, and Paper Trading

Objective: Close the learning loop.

Deliverables:

- Multi-account portfolio.
- Position ingestion.
- Risk dashboard.
- Greeks.
- Scenario stress.
- Journal.
- Automatic context snapshots.
- Paper broker.
- Simulated fills.
- Pre-trade guardrails.
- Strategy attribution.
- Behavioral pattern analysis.

Exit criteria:

- Accounting reconciles.
- Duplicate events/orders are safe.
- Risk limits are enforced in the backend.
- Journal can reconstruct the full trade context.

Phase 7 — Broker/Exchange Connectivity and Controlled Execution

Objective: Add secure, governed execution.

Deliverables:

- Read-only broker sync.
- Draft orders.
- User-approved live orders.
- Reconciliation.
- Partial-fill handling.

- Idempotency.
- Step-up auth.
- Kill switch.
- Daily loss limits.
- Approval workflow.
- Exchange adapter sandbox.
- Incident runbooks.

Exit criteria:

- Independent security review.
- Chaos tests for timeout/unknown order state.
- Audit completeness.
- Emergency disablement tested.
- No conversational AI direct-submit path.

Phase 8 — Enterprise, On-Prem, and Team Collaboration

Objective: Support professional teams.

Deliverables:

- Organizations.
- RBAC/ABAC.
- SSO.
- Team workspaces.
- Review/approval.
- Central data services.
- On-prem deployment.
- HSM/Vault.
- SIEM integration.
- Retention policy.
- Dedicated tenant options.
- Compliance exports.
- Enterprise support tooling.

Phase 9 — Ecosystem and Marketplace

Objective: Become an extensible platform.

Deliverables:

- Signed plugin SDK.
- Strategy packages.
- Indicator packages.
- MCP registry.
- Extension review.
- WASM sandbox.
- Marketplace governance.
- Revenue sharing.
- Reproducible package builds.
- Revocation and emergency disablement.

33. Team Model and Delivery Workstreams

Recommended workstreams:

53. Product and market domain.
54. Rust platform/core.
55. Market data and provider integrations.
56. Quantitative research.
57. Options analytics.
58. Frontend/design system.
59. GPU visualization.
60. AI/tooling.
61. Security/identity.
62. SRE/data operations.
63. QA/model validation.
64. Compliance/licensing.

For a small founding team, combine roles but preserve review separation for execution, cryptography, and model validation.

34. Acceptance Criteria and Product Metrics

34.1 User Value

- Time from question to evidence-backed answer.
- Opportunity review completion.
- Alert usefulness.
- Backtest-to-paper-trade conversion.
- Journal completion.
- Reduction in repeated mistakes.
- User trust in provenance.
- Workspace retention.

34.2 Data Quality

- Freshness.
- Completeness.
- Provider agreement.
- Schema failure.
- Identifier resolution.
- Event deduplication.
- Filing parsing accuracy.

34.3 Quant Quality

- Reproducibility.
- Calibration.
- Drift.
- False discovery.
- Slippage sensitivity.
- Out-of-sample performance.
- Strategy stability.

34.4 Reliability

- Crash-free sessions.
- API success.
- Alert delivery.
- Queue lag.
- Restore success.
- Update success.
- Order reconciliation.

34.5 Security

- Secrets leaked: zero.
- Unauthorized execution: zero.
- Critical unpatched vulnerabilities.
- Key rotation compliance.
- Signed artifact coverage.
- Plugin violations.
- Audit completeness.

35. Risk Register and Mitigations

35.1 Market Data Licensing

Risk: Data cannot legally be stored, displayed, or redistributed as assumed.
Mitigation: Entitlement registry, legal review, adapter-specific retention rules.

35.2 Free-Tier Dependency

Risk: Demo/free APIs are rate-limited or changed.
Mitigation: Cache, budget governor, paid upgrade path, provider abstraction.

35.3 False Predictive Confidence

Risk: Users interpret estimates as guarantees.
Mitigation: Calibration, intervals, evidence, warnings, model governance.

35.4 Backtest Overfitting

Risk: Strategies appear profitable due to leakage or repeated experimentation.
Mitigation: Holdouts, walk-forward, purging, multiple-hypothesis controls, immutable manifests.

35.5 Uncertain Option Flow Meaning

Risk: Flow is misclassified.
Mitigation: Fact/inference split, confidence, spread clustering, contradictory evidence.

35.6 Unauthorized Execution

Risk: Credential compromise or AI/tool abuse.
Mitigation: Dedicated execution boundary, step-up auth, approvals, scoped credentials, kill switch.

35.7 Plugin Supply Chain

Risk: Malicious or compromised extension.
Mitigation: Signing, sandbox, capabilities, review, revocation.

35.8 Data Poisoning

Risk: Bad data corrupts models or signals.
Mitigation: Provenance, cross-checks, quarantine, immutable snapshots.

35.9 Complexity

Risk: Product attempts too much too early.
Mitigation: Crypto MVP, options wedge, phase gates, modular monolith.

35.10 Regulatory Exposure

Risk: Product behavior crosses into regulated advice, brokerage, or automated trading obligations.
Mitigation: Specialized legal review, clear role definition, disclosures, licensing analysis, geographic controls.

36. Recommended Technology Decisions

Core

- Rust stable toolchain.
- Tokio.
- Axum.
- SQLx.
- Serde.
- thiserror/anyhow by layer.
- tracing/OpenTelemetry.
- reqwest.
- rust_decimal for authoritative monetary amounts where appropriate.
- Polars/Arrow.
- Rayon.
- Tauri 2.
- SvelteKit and Svelte 5.
- TypeScript strict mode.
- Valibot or Zod at UI boundaries.
- pnpm.
- wgpu and WGSL.
- PostgreSQL.
- ClickHouse.
- DuckDB.
- SQLite.
- NATS JetStream when durable event streaming is needed.
- Object storage compatible with S3 APIs.

Security

- OS keychain integration.
- secrecy.
- zeroize.
- rustls.
- Argon2id for password-derived keys where required.
- AES-256-GCM or XChaCha20-Poly1305 for authenticated local encryption.
- OIDC/passkeys.
- Vault/KMS/HSM for server keys.
- Sigstore/cosign for supply-chain artifacts where applicable.
- NIST-standardized PQC through maintained and verified implementations as ecosystem maturity allows.

Frontend

- Design-token package.
- CSS variables.
- Svelte transitions used sparingly.
- Web Animations API where appropriate.
- Canvas/WebGPU for dense visuals.
- SVG for accessible low-density graphics.
- Floating UI for overlays.
- Accessible headless primitives with careful dependency review.
- Playwright.
- Vitest.

37. Future Advancements Beyond the Initial Vision

37.1 Causal Market Graph

Move beyond correlation by representing hypothesized causal pathways among:

- Macro releases.
- Rates.
- Sectors.
- Companies.
- Options.
- Crypto.
- Institutional flows.
- Events.

The system must distinguish learned association from validated causality.

37.2 Market Digital Twin

Create a replayable market environment containing:

- Historical order/quote approximations.
- Events.
- Portfolios.
- Strategies.
- Agent behavior.
- Liquidity shocks.
- Correlation transitions.

37.3 Federated Strategy Learning

Allow organizations to improve models across isolated environments without centralizing raw strategy data, subject to rigorous privacy and security review.

37.4 Confidential Computing

Evaluate hardware-backed confidential VMs/enclaves for:

- Strategy execution.
- Sensitive model inference.
- Key operations.
- Shared institutional research.

37.5 Verifiable Research Artifacts

Produce signed result bundles that contain:

- Dataset hashes.
- Code hash.
- Model hash.
- Parameters.
- Environment.
- Metrics.
- Signature.

37.6 Natural-Language-to-Research Compiler

Convert a user thesis into:

65. Formal hypothesis.
66. Required data.
67. Bias checklist.
68. Strategy draft.
69. Validation plan.
70. Backtest.
71. Simulation.
72. Research report.

No automatic promotion to live trading.

37.7 Multi-Agent Research Council

Use specialized constrained agents:

- Filing analyst.
- Options analyst.
- Macro analyst.
- Crypto analyst.
- Quant reviewer.
- Risk critic.
- Data-quality reviewer.

A synthesis agent may summarize disagreements rather than forcing consensus.

37.8 Opportunity "Contrarian Mode"

Actively search for evidence that disproves the leading thesis and quantify fragility.

37.9 Privacy-Preserving Community Intelligence

Users may opt into aggregated, anonymized research statistics without sharing positions, identity, or proprietary strategy logic.

37.10 Strategy Lineage Graph

Track every strategy fork, parameter change, test, promotion, trade, and retirement.

37.11 Temporal Knowledge Graph

Maintain validity intervals so the platform can answer:

> **DESIGN PRINCIPLE** "What did the system know at 10:03 a.m. on that date?"

This is crucial for audit and look-ahead prevention.

37.12 Adaptive Interface

The workspace can adapt based on task and expertise, but all automated layout changes must be predictable, reversible, and user-controlled.

37.13 Resilient Multi-Provider Consensus

For critical fields, use quorum-like policies:

- Preferred source.
- Secondary verification.
- Tolerance.
- Conflict alert.
- No-trade state if disagreement exceeds policy.

38. Initial Backlog

Epic A — Repository and Governance

- Create workspace.
- Add standards.
- Add ADR templates.
- Add workbook.
- Add CI.
- Add SBOM.
- Add threat model.
- Add crypto inventory.

Epic B — Desktop Shell

- Tauri setup.
- Capability policy.
- CSP.
- Secure IPC.
- Window management.
- Auto updater.
- Crash recovery.
- Design system.

Epic C — CoinGecko Adapter

- Auth.
- Rate governor.
- Asset identity.
- Search.
- Global.
- Markets.
- Coin detail.
- Market chart.
- OHLC.
- Categories.
- Exchanges.
- Contract tests.
- Cache.

Epic D — Crypto UI

- Command dashboard.
- Watchlist.
- Asset workspace.
- Chart.
- Category map.
- Scanner.
- Alerts.
- Portfolio.

Epic E — Data Foundation

- PostgreSQL.
- SQLite.
- DuckDB.
- Parquet.
- Migrations.
- Encryption.
- Provenance.
- Data-quality service.

Epic F — Security

- Keychain.
- Secret types.
- Session policy.
- Audit.
- Redaction.
- Update signing.
- PQC exception register.
- Plugin trust model.

Epic G — Observability

- Tracing.
- Metrics.
- Health.
- Provider dashboard.
- Local diagnostics package.
- Privacy controls.

39. Reference Architecture Diagrams

39.1 Data Flow

MERMAID SOURCE

```
flowchart LR
    CG[CoinGecko] --> GW[Provider Gateway]
    UW[Unusual Whales] --> GW
    SEC[SEC EDGAR] --> GW
    CFTC[CFTC PRE] --> GW
    FRED[FRED] --> GW

    GW --> RAW[Raw / Immutable Layer]
    RAW --> NORM[Normalization]
    NORM --> QUALITY[Quality + Provenance]
    QUALITY --> STORE[(PostgreSQL / ClickHouse / Parquet)]
    STORE --> FEATURES[Feature Engine]
    FEATURES --> QUANT[Backtest / Simulation / Models]
    QUANT --> OPP[Opportunity Engine]
    OPP --> API[Rust Application API]
    API --> UI[Svelte / Tauri UI]
```

39.2 Execution Boundary

MERMAID SOURCE

```
flowchart TD
    UI[User Interface] --> DRAFT[Order Draft]
    AI[AI Analyst] --> DRAFT
    DRAFT --> CHECKS[Deterministic Pre-Trade Checks]
    CHECKS --> APPROVAL[User Approval + Step-Up Auth]
    APPROVAL --> EXEC[Execution Service]
    EXEC --> BROKER[Broker / Exchange]
    BROKER --> RECON[State Reconciliation]
    RECON --> AUDIT[Immutable Audit]
    RECON --> PORT[Portfolio]
```

39.3 Security Zones

MERMAID SOURCE

```
flowchart LR
    WEBVIEW[Svelte WebView<br/>No Secrets] -->|Validated IPC| CORE[Rust Trusted Core]
    CORE --> VAULT[OS Keychain / Vault]
    CORE --> DATA[(Encrypted Data)]
    CORE --> GATEWAY[Provider Gateway]
    CORE --> TOOL[AI Tool Gateway]
    TOOL --> MCP[Approved MCP Servers]
    CORE --> EXEC[Isolated Execution Boundary]
```

40. Source and Standards References

Product Engineering Standards

This design is intended to follow the supplied engineering standards emphasizing production readiness, typed contracts, idempotency, bounded concurrency, observability, secure defaults, testing, documentation, project workbooks, supply-chain security, and cryptographic agility.

Official External References

- CoinGecko API documentation: https://docs.coingecko.com/
- CoinGecko endpoint overview: https://docs.coingecko.com/reference/endpoint-overview
- CoinGecko rate-limit guidance: https://docs.coingecko.com/docs/common-errors-rate-limit
- CoinGecko data-delivery guidance: https://docs.coingecko.com/docs/data-delivery-methods
- Unusual Whales API documentation: https://api.unusualwhales.com/docs
- Unusual Whales public API: https://unusualwhales.com/public-api
- Unusual Whales MCP guidance: https://unusualwhales.com/public-api/mcp
- SEC EDGAR APIs: https://www.sec.gov/search-filings/edgar-application-programming-interfaces
- SEC developer resources: https://www.sec.gov/about/developer-resources
- CFTC Commitments of Traders: https://www.cftc.gov/MarketReports/CommitmentsofTraders/index.htm
- CFTC Public Reporting Environment guide: https://publicreporting.cftc.gov/stories/s/User-s-Guide/p2fg-u73y/
- NIST Post-Quantum Cryptography: https://csrc.nist.gov/projects/post-quantum-cryptography
- NIST FIPS 203, ML-KEM: https://csrc.nist.gov/pubs/fips/203/final
- NIST FIPS 204, ML-DSA: https://csrc.nist.gov/pubs/fips/204/final
- Tauri security: https://v2.tauri.app/security/
- Tauri capabilities: https://v2.tauri.app/security/capabilities/
- Svelte documentation: https://svelte.dev/docs/svelte
- wgpu: https://wgpu.rs/

Closing Direction

PRISMATIK should be built as a disciplined intelligence platform, not a decorative dashboard and not a black-box signal seller. Its strongest identity will come from combining:

- Broad but provenance-aware market data.
- Institutional and event context.
- Deep options intelligence.
- Credible quantitative validation.
- Transparent uncertainty.
- Secure local and enterprise deployment.
- A captivating but operationally responsible interface.
- A provider-neutral, Rust-centered architecture that can evolve for decades.

The first implementation should prove that architecture through CoinGecko-powered crypto intelligence, then add SEC/CFTC context, then establish the core options wedge with Unusual Whales, followed by quant validation, simulations, paper trading, and only later controlled execution.

MYTHOS SYSTEMS / PRISMATIK

08

BUILD-READY ARCHITECTURE CONTRACTS

The load-bearing traits, orchestration, lineage, cost model, onboarding, compliance, model serving, and migration contracts.

CORNERSTONE EXECUTIVE ARCHITECTURE REPORT | 2026

Architecture Evolution v0.2 - Integrated Build-Ready Contracts

INTEGRATION NOTE The following material preserves the complete Architecture Evolution v0.2 companion and integrates it into this cornerstone report as the implementation contract for strategy runtimes, risk, brokers, AI tools, data lineage, model serving, compliance, notifications, backup, and migration.

How to Use This Document

Each section maps to a gap in the v0.1 overview. They are ordered by implementation leverage, not document order. The Rust trait definitions in Part I are the single highest-priority addition — they are the actual architectural contract that every workstream in v0.1 §33 depends on.

Part I — Load-Bearing Trait Surfaces

DESIGN PRINCIPLE The v0.1 overview defines one Rust trait (MarketDataProvider) out of approximately twelve load-bearing interfaces. This part defines the rest. Each trait is written to be reviewable as a standalone ADR.

1. Strategy and Strategy Runtime

The v0.1 overview (§11.1) mandates a typed DSL that "MUST compile to a typed intermediate representation" but never defines the IR or the trait a strategy must implement to be runnable by the backtest engine, paper broker, and live execution boundary. This is the contract that makes a strategy portable across all three runtimes.

1.1 STRATEGY INTERMEDIATE REPRESENTATION

RUST

```rust
/// A compiled, type-checked strategy in intermediate representation form.
/// Produced by the DSL compiler, the visual builder codegen, or hand-written
/// Rust strategies via the SDK. All strategies — regardless of origin —
/// converge to this type before they touch a runtime.
#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct StrategyIR {
    pub id: StrategyId,
    pub name: String,
    pub version: SemanticVersion,
    pub universe: UniverseSpec,
    pub entry: Vec<EntryRule>,
    pub exit: Vec<ExitRule>,
    pub risk: RiskBinding,
    pub parameters: ParameterSet,
    pub capabilities: StrategyCapabilities,
    pub source_hash: ContentHash,
    pub compiled_at: OffsetDateTime,
    pub compiler_version: SemanticVersion,
}

/// What a strategy is allowed to do. The runtime enforces these.
/// A backtest strategy gets data access; a live strategy additionally
/// gets order draft access — but never direct submit.
#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct StrategyCapabilities {
    pub data_access: DataAccessScope,
    pub can_read_options: bool,
    pub can_read_filings: bool,
    pub can_read_crypto: bool,
    pub can_draft_orders: bool,
    pub can_access_network: bool,  // always false in backtest
    pub max_position_concentration: Option<Ratio>,
    pub max_premium_at_risk: Option<Ratio>,
}
```

1.2 THE STRATEGY TRAIT

RUST

```rust
/// The core trait every strategy implements, regardless of authoring mode.
/// The runtime calls `on_event` with a deterministic clock and scoped data.
/// Strategies are stateful but must be reproducible: same events + same
/// seed = same decisions.
#[async_trait::async_trait]
pub trait Strategy: Send + Sync {
    /// Unique identity for this strategy instance.
    fn id(&self) -> StrategyId;

    /// The compiled IR this instance was created from.
    fn ir(&self) -> &StrategyIR;

    /// Called once at the start of a run (backtest, paper, or live).
    /// Provides the deterministic clock, scoped data reader, and seed.
    async fn initialize(
        &mut self,
        ctx: &StrategyContext,
    ) -> Result<(), StrategyError>;
```

```
    /// Called for every event the strategy's universe produces.
    /// Market data, option prints, filings, and scheduled triggers
    /// all arrive as typed events. The strategy returns zero or more
    /// order intents — never orders directly.
    async fn on_event(
        &mut self,
        event: &MarketEvent,
        ctx: &StrategyContext,
    ) -> Result<Vec<OrderIntent>, StrategyError>;

    /// Called once at the end of a run. Allows the strategy to
    /// emit summary metadata, close state, and record final
    /// decisions for the journal and audit log.
    async fn finalize(
        &mut self,
        ctx: &StrategyContext,
    ) -> Result<StrategySummary, StrategyError>;
}

/// Scoped context passed to every strategy call. Prevents direct
/// access to the network, the broker, or data outside the strategy's
/// declared universe and capabilities.
pub struct StrategyContext {
    clock: DeterministicClock,
    data: ScopedDataReader,
    rng: SeededRng,
    run_id: RunId,
    manifest: ManifestBuilder,  // accumulates reproducibility metadata
}
```

1.3 WHY THIS DESIGN

- **One trait, three runtimes.** The backtest engine, paper broker, and live execution boundary all call `on_event`. The difference is what the `StrategyContext` permits and what happens to the returned `OrderIntent` — in backtest it is simulated, in paper it is recorded against the paper broker, in live it enters the deterministic execution workflow (v0.1 §17.5).
- **Capabilities are structural, not advisory.** `can_access_network` is enforced by the runtime, not by convention. A backtest cannot accidentally call a live API.
- **`OrderIntent`, not `Order`.** Strategies never produce orders. They produce intents that pass through risk checks (§3 below), user approval, and the execution boundary before becoming orders. This is the architectural enforcement of v0.1 §16.2: "Live order submission MUST not be directly available to a general conversational model."

2. Risk Policy and Pre-Trade Checks

The v0.1 overview (§17.3) lists 13 pre-trade checks but defines no trait, no evaluation model, and no fail-closed return type. Risk enforcement must be structural — a strategy or AI tool cannot bypass it by construction.

2.1 RISK TRAIT

RUST

```
/// A risk policy is a collection of checks evaluated before any order
/// leaves the system. Checks are short-circuiting: the first hard deny
/// stops evaluation. All checks (including those that passed) are
/// recorded in the audit log.
pub trait RiskPolicy: Send + Sync {
    /// Evaluate an order intent against all applicable checks.
    /// Returns a decision with the full evidence chain.
    fn evaluate(
        &self,
        intent: &OrderIntent,
        portfolio: &PortfolioState,
        ctx: &RiskContext,
    ) -> RiskDecision;
}

/// The decision returned by risk evaluation. Never silent.
#[derive(Clone, Debug, Serialize, Deserialize)]
pub enum RiskDecision {
    /// All checks passed. The intent may proceed to the execution boundary.
    Allow {
        checks_evaluated: Vec<CheckResult>,
    },
    /// A soft warning fired but did not block. The intent proceeds,
    /// but the warning is recorded and surfaced to the user.
    Warn {
        warning: CheckWarning,
        checks_evaluated: Vec<CheckResult>,
    },
```

```
    /// A hard deny. The intent is blocked. No further processing.
    /// The reason, the check that triggered, and the portfolio state
    /// at evaluation time are all captured for audit.
    Deny {
        reason: DenyReason,
        blocking_check: CheckId,
        portfolio_snapshot: PortfolioSnapshot,
        checks_evaluated: Vec<CheckResult>,
    },
}

/// Individual pre-trade check. Each is a pure function of
/// (intent, portfolio, context) -> CheckResult.
pub trait PreTradeCheck: Send + Sync {
    fn id(&self) -> CheckId;
    fn severity(&self) -> CheckSeverity;  // HardDeny | SoftWarn
    fn evaluate(
        &self,
        intent: &OrderIntent,
        portfolio: &PortfolioState,
        ctx: &RiskContext,
    ) -> CheckResult;
}
```

2.2 CHECK CATALOG

Mapping the 13 checks from v0.1 §17.3 to concrete implementations:

Check ID	Severity	Rule
max_position	HardDeny	Position notional > RiskPolicy.max_position
max_premium	HardDeny	Premium at risk > RiskPolicy.max_premium_pct of account
max_account_loss	HardDeny	Realized + unrealized loss > daily limit
sector_concentration	HardDeny	Sector exposure > RiskPolicy.max_sector_pct
correlation_cluster	SoftWarn	Adding position raises portfolio correlation > threshold
event_proximity	SoftWarn	Earnings/macro event within N hours
spread_width	SoftWarn	Bid-ask spread > RiskPolicy.max_spread_pct
liquidity_volume	SoftWarn	30-day avg volume < threshold
open_interest	SoftWarn	Open interest < threshold
account_permissions	HardDeny	Account not enabled for this asset/strategy type
market_status	HardDeny	Market closed, halted, or in pre/post with no permission
stale_data	HardDeny	Quote/filing/IV data older than RiskPolicy.max_data_age
duplicate_order	HardDeny	Idempotency key collision or same-contract intent within window

3. Broker Gateway and Execution Boundary

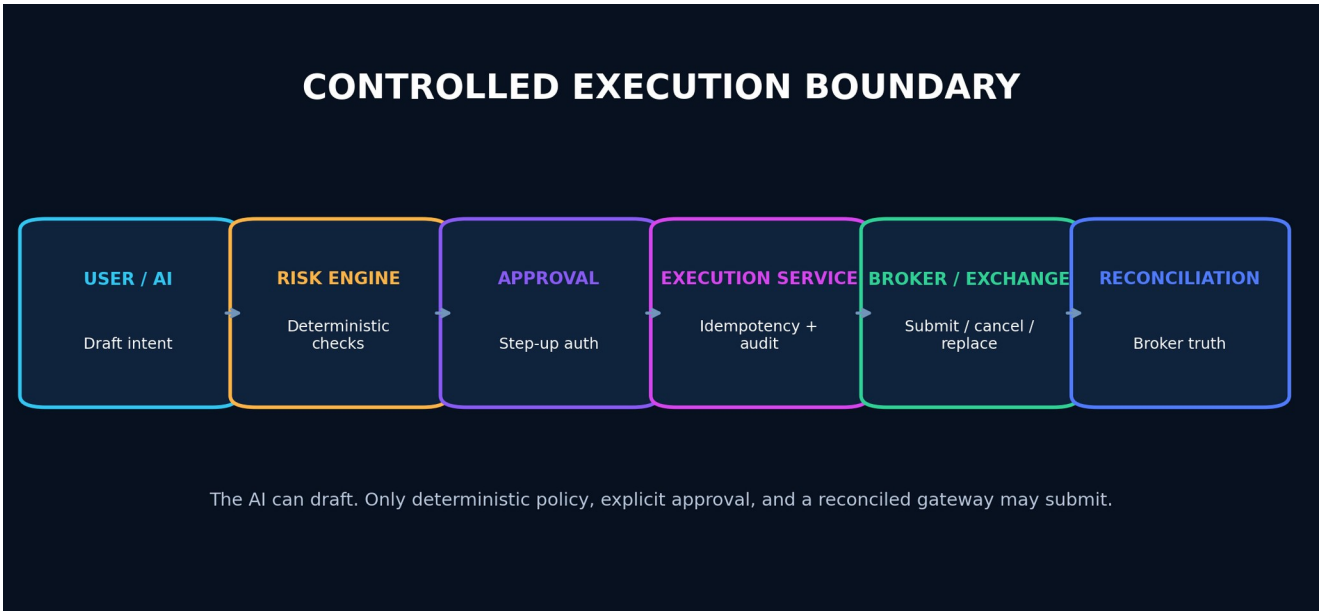

Figure 10 - The controlled execution boundary: AI and users draft; deterministic policy and explicit approval govern submission.

The v0.1 overview (§17.5, §39.2) mandates idempotency keys, reconciliation, and an isolated execution boundary, but defines no gateway trait. This is the contract between PRISMATIK and any broker or exchange.

RUST

```rust
/// The execution gateway abstraction. Broker adapters implement this.
/// The execution service wraps every call with idempotency, audit,
/// reconciliation, and the step-up auth gate from v0.1 §17.5.
#[async_trait::async_trait]
pub trait BrokerGateway: Send + Sync {
    fn broker_id(&self) -> BrokerId;
    fn capabilities(&self) -> BrokerCapabilities;

    /// Submit an order that has passed risk checks and user approval.
    /// Uses the idempotency key from the OrderIntent — a retry with
    /// the same key returns the original result, never creates a duplicate.
    async fn submit(
        &self,
        order: ApprovedOrder,
    ) -> Result<SubmissionResult, BrokerError>;

    /// Cancel a working order. Must be idempotent.
    async fn cancel(
        &self,
        order_id: BrokerOrderId,
    ) -> Result<CancelResult, BrokerError>;

    /// Replace a working order (modify quantity, price, or stop).
    /// Not all brokers support this. Check capabilities first.
    async fn replace(
        &self,
        order_id: BrokerOrderId,
        modification: OrderModification,
    ) -> Result<ReplaceResult, BrokerError>;

    /// Reconcile local state with broker state. Called after
    /// timeouts, unknown results, disconnections, and periodically
    /// as a safety net. Returns the delta between local belief
    /// and broker truth.
    async fn reconcile(
        &self,
        local_state: &LocalOrderState,
    ) -> Result<ReconciliationDelta, BrokerError>;

    /// Stream fills and order state changes.
    async fn stream_events(
        &self,
    ) -> BoxStream<'static, BrokerEvent>;
}
```

4. AI Tool Gateway

The v0.1 overview (§16.2, §31.5) lists 11 controlled tools and 10 gateway enforcement points, but the dispatch trait is undefined. This is the boundary between the conversational AI and the platform's controlled surface area.

```rust
/// The AI tool gateway. Every tool the AI can invoke passes through
/// this trait. The gateway enforces schema validation, rate limits,
/// approval hooks, output filtering, and audit logging — not the tool.
#[async_trait::async_trait]
pub trait ToolGateway: Send + Sync {
    /// Dispatch a tool call from the AI. The gateway validates
    /// the input schema, checks rate limits, optionally requests
    /// human approval, executes the tool, filters the output,
    /// and returns a controlled result.
    async fn dispatch(
        &self,
        call: ToolCall,
        ctx: &ToolContext,
    ) -> Result<ToolResult, ToolError>;

    /// List the tools available to the current session,
    /// respecting trust level and capability scope.
    fn available_tools(&self, trust: TrustLevel) -> Vec<ToolDescriptor>;
}

/// A single tool in the gateway registry.
pub trait ControlledTool: Send + Sync {
    fn descriptor(&self) -> ToolDescriptor;
    fn trust_level(&self) -> TrustLevel;  // ReadOnly | DraftOnly | Sensitive
    fn requires_approval(&self) -> bool;

    async fn execute(
        &self,
        input: ToolInput,
        ctx: &ToolContext,
    ) -> Result<ToolOutput, ToolError>;
}
```

TOOL CATALOG WITH TRUST LEVELS

Tool	Trust Level	Approval?	Maps to v0.1 §16.2
market_query	ReadOnly	No	Read-only market query
filing_query	ReadOnly	No	Filing query
cot_query	ReadOnly	No	COT query
backtest_request	ReadOnly	No	Backtest request
simulation_request	ReadOnly	No	Simulation request
chart_snapshot	ReadOnly	No	Chart snapshot
portfolio_analysis	ReadOnly	No	Portfolio analysis
journal_search	ReadOnly	No	Journal search
draft_alert	DraftOnly	No	Draft alert
draft_strategy	DraftOnly	No	Draft strategy
draft_order	Sensitive	Yes — user must approve	Draft order

Live order submission is intentionally absent from this table. It is not a tool. It is a deterministic workflow (v0.1 §17.5) that the `draft_order` tool feeds into, but the AI never calls submit directly.

5. Alert Rule Evaluator

The v0.1 overview (§8.8) lists 12 alert targets and 9 delivery channels but defines no rule schema, no evaluator trait, and no streaming-vs-polling boundary.

```rust
/// An alert rule. Compiled from the rule DSL into a typed, evaluable form.
/// Rules are either streaming (evaluated on every event) or scheduled
/// (evaluated on a cron-like trigger).
pub trait AlertRule: Send + Sync {
    fn id(&self) -> AlertRuleId;
    fn evaluation_mode(&self) -> EvaluationMode;  // Streaming | Scheduled
    fn dedup_key(&self, event: &MarketEvent) -> Option<DedupKey>;

    /// Evaluate against the current state. Returns Some(AlertFiring)
    /// if the condition is met, None otherwise.
    fn evaluate(&self, state: &AlertContext) -> Option<AlertFiring>;
}

pub enum EvaluationMode {
    /// Evaluated on every incoming market event. Used for price,
    /// volume, IV, and flow conditions.
    Streaming,
    /// Evaluated on a schedule. Used for filings, COT releases,
    /// and daily risk summaries.
    Scheduled(Schedule),
}
```

STREAMING VS SCHEDULED BOUNDARY

Condition	Mode	Rationale
Price threshold	Streaming	Sub-second relevance
Technical indicator	Streaming	Recomputed on every tick/bar
Volume / relative volume	Streaming	Intraday signal
Options flow	Streaming	Print-level detection
Implied volatility	Streaming	Updated with chain refresh
Filing arrival	Scheduled	SEC RSS poll or webhook
COT extreme	Scheduled	Weekly data, no intraday value
Event proximity	Scheduled	Calendar-driven
Portfolio risk threshold	Scheduled	Recompute on fill + periodic

6. Repository Port Pattern

The v0.1 overview (§18.3) lists "Persistence" as a Rust responsibility but defines no repository traits, leaving domain crates coupled to concrete stores. This is the hexagonal port that keeps the domain pure.

```rust
/// Generic repository port. Domain crates depend on this trait,
/// never on SQLx or a concrete store. The application layer wires
/// concrete implementations (PostgresUserRepository, SqliteUserRepository).
#[async_trait::async_trait]
pub trait Repository<T: Aggregate>: Send + Sync {
    async fn get(&self, id: &T::Id) -> Result<Option<T>, RepoError>;
    async fn save(&self, aggregate: &T) -> Result<(), RepoError>;
    async fn delete(&self, id: &T::Id) -> Result<(), RepoError>;
    async fn exists(&self, id: &T::Id) -> Result<bool, RepoError>;
}

/// Marker trait for domain aggregates that can be persisted.
pub trait Aggregate: Send + Sync + Clone {
    type Id: Clone + Send + Sync;
    fn id(&self) -> Self::Id;
    fn version(&self) -> AggregateVersion;  // optimistic concurrency
}
```

Part II — Pipeline Orchestration and Lineage

> **DESIGN PRINCIPLE** The v0.1 overview (§10.1, §22.1) defines six data layers and lists bus candidates, but never specifies the orchestrator, lineage schema, invalidation model, or freshness targets. This part fills those gaps.

7. Orchestrator Architecture

7.1 DESIGN DECISION

For the desktop deployment (Phase 0–1), use an **in-process Tokio task graph**. No external orchestrator (Temporal, Airflow) is needed for a single-user local-first application. The task graph persists its state to SQLite so it survives restarts.

For the enterprise deployment (Phase 8), evaluate **Temporal** for durable cross-service workflows, keeping the same `TaskRunner` trait so the domain logic is portable.

7.2 CORE TYPES

RUST

```rust
/// A pipeline task. Idempotent: running it twice with the same inputs
/// produces the same result (or a no-op if already complete).
#[async_trait::async_trait]
pub trait PipelineTask: Send + Sync {
    fn id(&self) -> TaskId;
    fn task_type(&self) -> TaskType;
    fn input_hash(&self) -> ContentHash;

    async fn run(&self, ctx: &PipelineContext) -> Result<TaskOutput, TaskError>;

    /// Whether this task can be safely retried after a partial failure.
    fn idempotent(&self) -> bool { true }
}

/// The task graph. Tasks declare their dependencies; the scheduler
/// resolves execution order. State persists to SQLite for crash recovery.
pub struct TaskGraph {
    tasks: Vec<TaskNode>,
    edges: Vec<(TaskId, TaskId)>,  // (dependency, dependent)
}

pub struct TaskNode {
    task: Box<dyn PipelineTask>,
    trigger: Trigger,
    status: TaskStatus,
    last_run: Option<OffsetDateTime>,
    last_output: Option<TaskOutput>,
}

pub enum Trigger {
    /// Runs on a schedule (cron-like).
    Scheduled(Schedule),
    /// Runs when an upstream task completes.
    Dependency(TaskId),
    /// Runs when new data arrives from a provider.
    DataEvent(ProviderId, EventPattern),
    /// Runs on explicit user request.
    OnDemand,
}
```

7.3 TASK TYPES MAPPED TO V0.1 LAYERS

Task Type	Input Layer	Output Layer	Example
Ingest	(external)	Raw	Fetch CoinGecko markets snapshot
Normalize	Raw	Normalized	Parse CoinGecko JSON → canonical MarketSnapshot

Task Type	Input Layer	Output Layer	Example
QualityCheck	Normalized	Curated	Score freshness, detect outliers, deduplicate
FeatureCompute	Curated	Feature	Compute RSI, IV rank, relative volume
IntelligenceCompute	Feature	Intelligence	Score opportunity, run model prediction
Materialize	Curated/Feature	Presentation	Build user-specific watchlist view
Backfill	(external)	Raw	Fetch historical OHLC for a new asset

8. Lineage Schema

Every record in the curated, feature, and intelligence layers carries lineage back to its raw sources. This is the enforcement of v0.1 §4.1 ("Evidence Before Conclusion") at the data level.

```rust
/// Attached to every curated, feature, and intelligence record.
/// Allows tracing any derived value back to its raw source(s).
#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct Lineage {
    /// The raw record IDs this record was computed from.
    pub upstream_record_ids: Vec<RecordId>,
    /// The transform that produced this record.
    pub transform_id: TransformId,
    /// The version of the transform code.
    pub transform_version: SemanticVersion,
    /// When this record was computed.
    pub computed_at: OffsetDateTime,
    /// Hash of all upstream record IDs + transform + version.
    /// If any input changes, this hash changes, and the record
    /// is considered stale.
    pub inputs_hash: ContentHash,
}
```

9. Invalidation Model

When a raw record is corrected (e.g., a restated 13F, a corrected macro series, a CoinGecko metadata fix), the system must determine which downstream records are affected and whether they need recomputation.

9.1 INVALIDATION PROPAGATION

```
Raw record corrected
    |
    ▼
Find all curated records with this raw ID in upstream_record_ids
    |
    ▼
Mark those curated records as STALE (do not delete — preserve for audit)
    |
    ▼
Find all feature records depending on those curated records
    |
    ▼
Mark feature records as STALE
    |
    ▼
Find all intelligence records (opportunities, predictions) depending on those features
    |
    ▼
Mark intelligence records as STALE
    |
    ▼
Re-trigger the pipeline tasks that produced the stale records
    |
    ▼
New records created with new lineage; old records retained for audit
```

9.2 STALE-VS-SOURCE VS RECOMPUTE

Not every correction requires recomputation:

Correction Type	Action	Reason
Metadata fix (name, description)	Recompute downstream	Low cost, high correctness value
Price/bar correction	Recompute + flag affected backtests	Critical — backtest results may change
Filing amendment	Recompute intelligence	Opportunities may be invalidated
COT revision	Recompute positioning metrics	Crowding scores change
Delayed-but-unchanged data	No action	Same data, later arrival — no invalidation

10. Per-Layer Freshness Targets

The v0.1 overview (§26.3) lists "Critical provider freshness" as an SLO but does not map targets per dataset. These are the targets for the Phase 0/1 data set:

Dataset	Layer	Target Freshness	Degradation Behavior
CoinGecko market snapshot (prices, cap, volume)	Raw	≤ 60 seconds	Show "delayed" badge at 120s; stop scoring at 300s
CoinGecko trending	Raw	≤ 5 minutes	Acceptable to be stale; no degradation
CoinGecko metadata	Raw	≤ 24 hours	Very low change rate; no degradation
CoinGecko OHLC (historical)	Raw	Immutable after finalization	N/A — never goes stale
SEC filings RSS	Raw	≤ 5 minutes after publication	Critical for event trading
CFTC COT	Raw	≤ 1 day after Tuesday release	Weekly data; no intraday value
FRED macro	Raw	≤ 1 day after release	Quarterly/monthly cadence
Normalized market snapshot	Normalized	≤ 5 seconds behind raw	Pipeline must be fast
Feature: RSI / indicators	Feature	≤ 10 seconds behind normalized	Recomputed on each bar close
Feature: IV rank	Feature	≤ 30 seconds behind normalized	Depends on chain refresh
Intelligence: opportunity score	Intelligence	≤ 30 seconds behind feature	Recomputed when inputs change

Part III — Provider Cost Model

> **DESIGN PRINCIPLE** The v0.1 overview (§10.5) lists what the budget governor MUST do but defines no cost unit, no budget policy, and no UI. This part makes it concrete.

11. Cost Unit and Budget Architecture

11.1 COST UNITS

Providers charge in different units. The governor normalizes all of them to a common `CostUnit` for tracking, while preserving the native unit for display.

RUST

```rust
/// The native cost unit of a provider.
pub enum CostUnit {
    /// API calls (SEC EDGAR, FRED — free but rate-limited).
    ApiCalls,
    /// Credits (CoinGecko paid tiers — credits per endpoint).
    Credits,
    /// Plan-based (Unusual Whales — flat plan, soft rate limits).
    PlanIncluded,
    /// Dollars (paid per-call providers in later phases).
    Dollars,
}

/// A single provider's cost model.
pub struct ProviderCostModel {
    pub provider_id: ProviderId,
    pub unit: CostUnit,
    pub period: BudgetPeriod,           // Hourly | Daily | Monthly
    pub budget: u64,                     // calls, credits, or cents
    pub remaining: u64,
    pub reservation_pool: u64,           // reserved for critical workflows
    pub endpoints: Vec<EndpointWeight>,  // credit weight per endpoint
}

pub struct EndpointWeight {
    pub endpoint: EndpointPath,
    pub weight: u32,   // e.g., CoinGecko /coins/{id} = 1 credit, /coins/markets = 1, historical = 5
}
```

11.2 BUDGET POLICY AND PRIORITY CLASSES

RUST

```rust
/// Workflows are classified by priority. When the budget is under
/// pressure, lower-priority requests are shed first.
pub enum PriorityClass {
    /// User-facing dashboard refresh, alert evaluation, live execution.
    /// These always run, even if they exhaust the budget.
    Critical,
    /// Watchlist updates, scanner runs, chart data.
    /// Run unless budget is below 20% remaining.
    Standard,
    /// Background data refresh, historical backfill, model retraining.
    /// Run only when budget is above 50% remaining.
    Background,
    /// Prefetch, warm cache, pre-compute.
    /// Run only when budget is above 80% remaining.
    Opportunistic,
}
```

11.3 RATE-BUDGET UI SPECIFICATION

The Phase 0 exit criteria require a "Provider health and rate-budget UI." This is what it shows:

Panel 1 — Provider Health Card (one per connected provider):

- Provider name and plan tier
- Current status: Healthy / Degraded / Rate-Limited / Down

- Requests in current period / budget
- Credits remaining (for credit-based providers)
- Average latency (p50, p95)
- Error rate
- Last successful request timestamp
- Next reset time

Panel 2 — Spend Trend:

- Sparkline of request/credit consumption over the current period
- Projected burn rate (linear extrapolation)
- "At current rate, budget exhausted in X hours" warning

Panel 3 — Priority Breakdown:

- Bar chart of requests by PriorityClass (Critical / Standard / Background / Opportunistic)
- Shed count (how many Background/Opportunistic requests were rejected)

Panel 4 — Endpoint Breakdown (expandable per provider):

- Table: endpoint path, call count, credit weight, total credits consumed

Controls:

- Per-provider pause/resume toggle
- Per-provider budget override (with confirmation)
- "Reserve N credits for critical workflows" slider

Part IV — First-Run, Demo, and Suitability

> **DESIGN PRINCIPLE** The v0.1 overview has no onboarding architecture despite Phase 0 requiring "CoinGecko demo integration" and §17.3 requiring "user-specific suitability filters."

12. First-Run Experience

12.1 FLOW STATE MACHINE

```
CODE
[Launch]
    │
    ▼
[Welcome Screen]
    │
    ├── "Get Started" ──────────────────▶ [Provider Setup]
    │                                        │
    │                                        ├── Enter CoinGecko key (optional)
    │                                        │   (demo key pre-filled; user can upgrade later)
    │                                        │
    │                                        └── "Skip — use demo mode" ──▶ [Demo Mode Active]
    │
    ▼
[Suitability Profile]
    │
    ├── Experience level: Novice / Intermediate / Advanced / Professional
    ├── Risk tolerance: Conservative / Moderate / Aggressive
    ├── Jurisdiction: Country + State (for feature gating)
    ├── Primary asset interest: Crypto / Equities / Options / Multi-asset
    └── Time horizon: Scalping / Day / Swing / Position / Long-term
    │
    ▼
[Default Workspace Seeded]
    │
    ├── Watchlist: BTC, ETH, SPY, QQQ (+ top 10 by user interest)
    ├── Sample strategy: "EventFlowMomentum" (read-only, for learning)
    ├── Risk policy: Conservative defaults (max 2% per position, max 15% sector)
    └── Layout: "First-Run" named layout with Command Center + one chart + watchlist
    │
    ▼
[Guided Tour (skippable)]
    │
    ├── "This is your Command Center. Cards show..."
    ├── "Every card links to its evidence. Click any source."
    ├── "Probabilities are ranges, not predictions. See how..."
    └── "Your risk limits are set. Here's how to change them."
    │
    ▼
[Ready — Main Application]
```

12.2 DEMO MODE ARCHITECTURE

Demo mode allows full exploration without API keys. It is not a separate app — it is a degraded-capability mode that is transparent to the user.

```rust
RUST
pub struct DemoMode {
    /// Pre-recorded provider responses, replayed deterministically.
    /// Recorded from CoinGecko demo endpoints with a known timestamp.
    fixtures: FixtureStore,
    /// The "simulation clock" — demo data appears to advance in real-time
    /// but is actually replayed from recorded snapshots at 1-minute granularity.
    sim_clock: DemoClock,
    /// Watermark shown on all data: "DEMO — Data as of 2026-07-10"
    watermark: DemoWatermark,
    /// Features disabled in demo: live execution, real broker sync, alerts to SMS.
    disabled_features: Vec<FeatureGate>,
}
```

Demo mode rules:

- Market data shows a watermark on every chart and table: "DEMO — Data as of [date]"
- No data is older than 7 days at time of recording
- Alerts fire against demo data but cannot deliver to SMS, email, or webhooks
- Paper trading works fully against demo data
- The user can enter real API keys at any time to transition out of demo mode seamlessly
- No real credentials are ever stored during demo mode

12.3 SUITABILITY PROFILE AGGREGATE

```rust
/// Captured during first-run and editable in settings.
/// Consumed by the scanner (filters out unsuitable strategies),
/// the opportunity engine (ranks by user fit), and the compliance
/// layer (gates features by jurisdiction and experience).
pub struct SuitabilityProfile {
    pub experience_level: ExperienceLevel,
    pub risk_tolerance: RiskTolerance,
    pub jurisdiction: Jurisdiction,
    pub primary_interests: Vec<AssetClass>,
    pub time_horizon: TimeHorizon,
    pub acknowledged_disclosures: Vec<DisclosureAcknowledgment>,
    pub created_at: OffsetDateTime,
    pub updated_at: OffsetDateTime,
}
```

Part V — Notification Service

DESIGN PRINCIPLE The v0.1 overview (§8.8) lists alert targets and delivery channels with no service architecture.

13. Notification Service Design

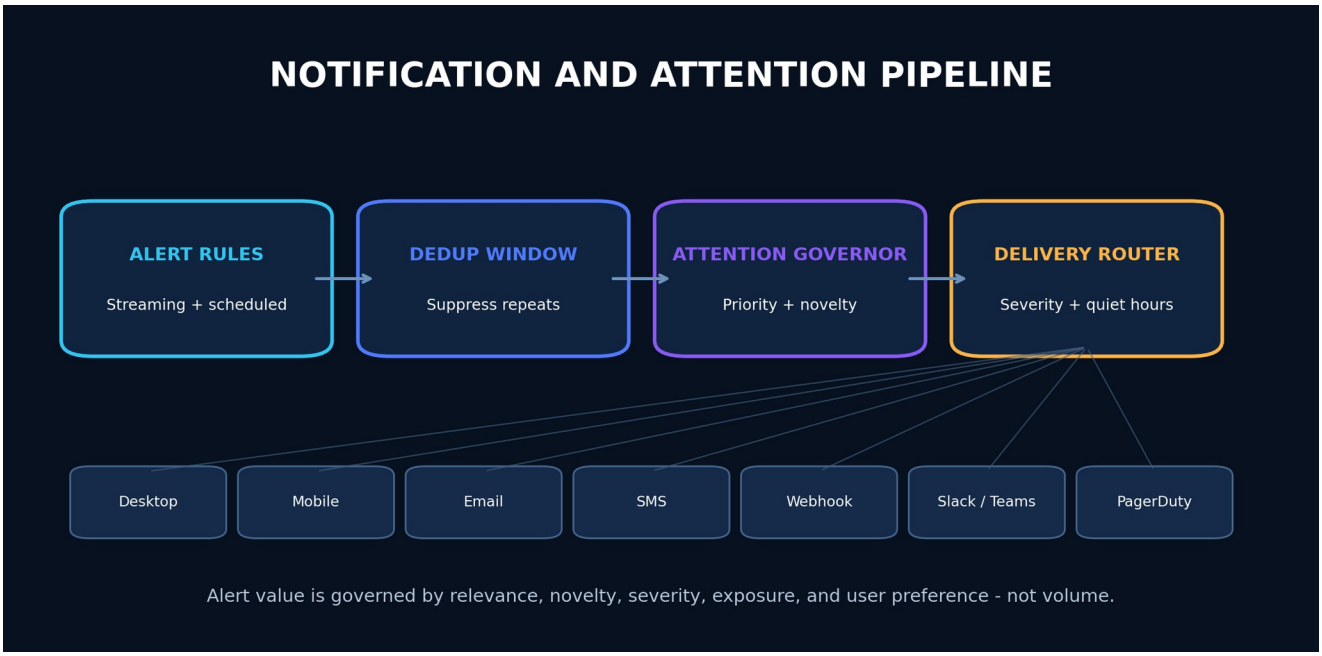

Figure 11 - Notification pipeline with deduplication, attention governance, severity routing, and quiet-hour controls.

CODE

```
┌──────────────────────────────────────────────────┐
│                  Alert Rules                     │
│  (Streaming evaluator + Scheduled evaluator)     │
└──────────────────────┬───────────────────────────┘
                       │ AlertFiring
                       ▼
┌──────────────────────────────────────────────────┐
│                  Dedup Window                    │
│  Key = (rule_id, entity, direction, severity)    │
│  If duplicate within window → suppress           │
│  If same entity, higher severity → escalate      │
└──────────────────────┬───────────────────────────┘
                       │ Unique AlertFiring
                       ▼
┌──────────────────────────────────────────────────┐
│               Attention Governor                 │
│  Prioritizes based on: portfolio exposure, watchlist
│  relevance, strategy fit, event proximity, severity,
│  novelty, historical usefulness.                 │
│  Limits repeated low-value alerts.               │
└──────────────────────┬───────────────────────────┘
                       │ Governed AlertFiring
                       ▼
┌──────────────────────────────────────────────────┐
│                Delivery Router                   │
│  Routes to user-preferred channels by severity and time.
│  Quiet hours suppress non-critical.              │
│  Critical alerts bypass quiet hours.             │
└──────────────────────────────────────────────────┘
    ▼      ▼      ▼      ▼      ▼      ▼      ▼
 Desktop Mobile Email  SMS  Webhook Slack  PagerDuty
 Toast   Push                 /Teams
```

13.1 DELIVERY CHANNEL ABSTRACTION

```rust
#[async_trait::async_trait]
pub trait DeliveryChannel: Send + Sync {
    fn channel_id(&self) -> ChannelId;
    fn min_severity(&self) -> AlertSeverity;

    async fn deliver(
        &self,
        alert: &AlertFiring,
        user_prefs: &NotificationPreferences,
    ) -> Result<DeliveryReceipt, DeliveryError>;

    /// Whether this channel is available (has credentials, is connected).
    async fn is_available(&self) -> bool;
}
```

13.2 DEDUP AND ESCALATION

Scenario	Action
Same rule fires for same entity within dedup window	Suppress (count incremented on original)
Same entity, higher severity than existing alert	Escalate (replace original, notify)
Rule fires for entity already alerted → condition clears → refires	New alert (dedup window reset on clear)
User dismisses alert; condition persists	Snooze for user-set duration, then re-fire if still active
User disables rule	No alerts fire; rule state preserved for re-enable
Quiet hours active, severity < Critical	Queue for delivery at quiet-hours end
Quiet hours active, severity = Critical	Deliver immediately, bypass quiet hours

Part VI — Compliance Architecture

> **DESIGN PRINCIPLE** The v0.1 overview (§35.10) treats compliance as legal review. This makes it architecture.

14. Jurisdiction Gating

```rust
/// Determined at first-run and verified periodically via IP check
/// (server-side for cloud deployments) or self-declaration (desktop).
/// Gates features that are legally restricted by jurisdiction.
pub struct JurisdictionGate {
    pub jurisdiction: Jurisdiction,
    pub feature_overrides: HashMap<FeatureId, FeatureState>,
    pub last_verified: OffsetDateTime,
}

pub enum FeatureState {
    Enabled,
    Disabled { reason: String },
    RequiresDisclosure { disclosure_id: DisclosureId },
}
```

Example gating rules:

Feature	US	EU	UK	Restricted jurisdictions
Crypto market data	Enabled	Enabled	Enabled	Disabled if under OFAC sanctions
Options flow	Enabled	Requires disclosure	Requires disclosure	Disabled
Congressional trading data	Enabled	Disabled (data licensing)	Disabled	Disabled
Live broker execution	Enabled (US brokers)	Enabled (EU brokers)	Enabled (UK brokers)	Disabled
AI investment analysis	Requires disclaimer	Requires MiDIS disclaimer	Requires FCA disclaimer	Disabled

15. Disclosure Registry

```rust
/// Every disclosure in the system is registered with an ID,
/// context (where it appears), text, and version. When a
/// disclosure is shown to a user, an audit event records it.
pub struct Disclosure {
    pub id: DisclosureId,
    pub context: DisclosureContext,  // Onboarding | OpportunityCard | RiskPanel | AIResponse | ...
    pub text: DisclosureText,
    pub version: SemanticVersion,
    pub jurisdiction: Option<Jurisdiction>,  // None = all jurisdictions
    pub required_acknowledgment: bool,
}

/// When a disclosure is rendered, this event is emitted.
/// It proves the user saw the disclosure for audit and compliance.
pub struct DisclosureShownEvent {
    pub disclosure_id: DisclosureId,
    pub user_id: UserId,
    pub shown_at: OffsetDateTime,
    pub context: DisclosureContext,
    pub acknowledged: bool,
}
```

Part VII — Model Serving and Feature Store

> **DESIGN PRINCIPLE** The v0.1 overview (§12) thoroughly covers model governance but has no runtime serving or feature store architecture.

16. Feature Store

The single most important property of a feature store for a trading platform is **point-in-time correctness** — a feature value used in training or inference must reflect exactly what was known at that timestamp, with no look-ahead.

```rust
/// Point-in-time feature query. Returns the feature value as it
/// existed at `as_of`, not the current value. This is the
/// mechanism that prevents look-ahead bias in backtests and
/// ensures reproducibility in live inference.
pub struct FeatureQuery {
    pub feature_view: FeatureViewId,
    pub entity_keys: Vec<EntityKey>,
    pub as_of: OffsetDateTime,        // the point in time
    pub feature_names: Vec<String>,    // which features to return
}

pub struct FeatureView {
    pub id: FeatureViewId,
    pub entities: Vec<EntityKeySchema>,
    pub features: Vec<FeatureSchema>,
    pub online_store: OnlineStore,    // SQLite (desktop) / Redis (enterprise)
    pub offline_store: OfflineStore,  // Parquet/DuckDB
    pub materialization: MaterializationSchedule,
    pub version: SemanticVersion,
}
```

16.1 ONLINE VS OFFLINE SERVING

Store	Used By	Latency Target	Data
Online (SQLite / Redis)	Live inference, streaming alerts, real-time scoring	≤ 5ms	Latest feature values, ~7 day window
Offline (Parquet / DuckDB)	Backtest, model training, batch research	Seconds acceptable	Full history, point-in-time queryable

16.2 MATERIALIZATION

Features are computed by pipeline tasks (Part II) and written to both stores. The online store retains only the latest value per entity. The offline store retains the full time-series with lineage.

17. Model Serving

```rust
/// A registered model in the model registry.
pub struct ModelArtifact {
    pub id: ModelId,
    pub name: String,
    pub version: SemanticVersion,
    pub artifact_uri: String,          // local path or object storage
    pub content_hash: ContentHash,
    pub signature: ModelSignature,     // input/output schema
    pub runtime: ModelRuntime,         // Onnx | LlamaCpp | CloudApi
    pub owner: UserId,
    pub validation_report: ValidationReportId,
    pub calibration_report: Option<CalibrationReportId>,
    pub status: ModelStatus,           // Draft | Validated | Promoted | Retired
}
```

```
/// Inference trait. The runtime binds the model version to every
/// prediction so that v0.1 §12.4's "model version" requirement
/// is enforced structurally.
#[async_trait::async_trait]
pub trait ModelRuntime: Send + Sync {
    async fn predict(
        &self,
        features: FeatureSet,
    ) -> Result<Prediction, ModelError>;

    /// Metadata bound to every prediction for audit and reproducibility.
    fn metadata(&self) -> ModelMetadata;
}

pub struct Prediction {
    pub model_id: ModelId,
    pub model_version: SemanticVersion,
    pub value: PredictionValue,         // Distribution | Probability | Ranking
    pub confidence_interval: Option<Interval>,
    pub feature_contributions: Vec<FeatureContribution>,  // explainability
    pub training_window: DateRange,
    pub drift_status: DriftStatus,
    pub predicted_at: OffsetDateTime,
}
```

Part VIII — Reproducibility Manifest Schema

DESIGN PRINCIPLE The v0.1 overview mandates "signed result manifests" in sections 4.7, 11, 37.5, and Phase 4 exit criteria, but never defines the format.

18. Manifest Schema

JSON

```
{
  "manifest_type": "backtest",
  "manifest_version": 1,
  "manifest_id": "01J...",
  "created_at": "2026-07-13T15:00:00Z",
  "created_by": "user_id",

  "dataset": {
    "snapshot_id": "snap_01J...",
    "snapshot_hash": "sha256:...",
    "as_of": "2026-07-10T00:00:00Z",
    "adjustment_policy": "TOTAL_RETURN",
    "calendar_version": "2026.1",
    "provider_versions": {
      "coingecko": "2026-07-10",
      "sec_edgar": "2026-07-10"
    }
  },

  "strategy": {
    "strategy_id": "strat_01J...",
    "strategy_hash": "sha256:...",
    "strategy_version": "1.2.0",
    "compiler_version": "0.3.0",
    "parameters": { "rsi_period": 14, "iv_rank_max": 45 }
  },

  "execution_assumptions": {
    "commission": "0.65 per contract",
    "slippage_model": "quarter_spread",
    "fill_assumption": "market_open",
    "partial_fill_policy": "proportional"
  },

  "random_seed": 42,

  "code": {
    "commit_hash": "abc123def456",
    "dependency_hash": "sha256:Cargo.lock hash",
    "build_flags": []
  },

  "metrics": {
    "total_return": 0.234,
    "sharpe": 1.82,
    "max_drawdown": -0.087,
    "win_rate": 0.61
  },

  "signatures": [
    {
      "algorithm": "ML-DSA",
      "key_id": "key_01J...",
      "signature": "base64..."
    },
    {
      "algorithm": "Ed25519",
      "key_id": "key_01J...",
      "signature": "base64..."
    }
  ]
}
```

VERIFICATION PROTOCOL

To verify a result reproduces from its manifest:

73. Reconstruct the dataset snapshot from `snapshot_hash`.
74. Load the strategy from `strategy_hash` (content-addressed).
75. Check out code at `commit_hash`.
76. Verify `dependency_hash` matches `Cargo.lock`.
77. Run the backtest with the same parameters, seed, and execution assumptions.
78. Compare metrics within the documented numeric tolerance.
79. Verify both signatures against the trusted key store.

Part IX — Market Infrastructure Services

> **DESIGN PRINCIPLE** The v0.1 overview requires trading calendars, corporate actions, and symbology (§8.3, §10.3, §11.2) but names no sources or services.

19. Trading Calendar Service

```rust
/// Provides session boundaries, holidays, and half-days per venue.
/// Used by the backtest engine (replay only on valid sessions),
/// the execution boundary (block orders when market closed),
/// and the chart (show session breaks).
pub trait TradingCalendar: Send + Sync {
    fn is_open(&self, venue: Venue, at: OffsetDateTime) -> bool;
    fn next_open(&self, venue: Venue, from: OffsetDateTime) -> OffsetDateTime;
    fn next_close(&self, venue: Venue, from: OffsetDateTime) -> OffsetDateTime;
    fn session_for(&self, venue: Venue, at: OffsetDateTime) -> Option<Session>;
    fn holidays(&self, venue: Venue, year: i32) -> Vec<Holiday>;
}
```

Source: Self-hosted calendar data generated from exchange-published schedules, stored as versioned JSON, updated via signed data patches. Version pinned in reproducibility manifests.

20. Corporate Action Service

```rust
/// Corporate actions that affect price series and backtest integrity.
/// Every action specifies how it propagates to historical bars.
pub struct CorporateAction {
    pub id: CorporateActionId,
    pub asset_id: AssetId,
    pub action_type: ActionType,       // Split | Dividend | Spinoff | Merger | Delisting
    pub ex_date: Date,
    pub record_date: Option<Date>,
    pub payable_date: Option<Date>,
    pub ratio: Option<Ratio>,          // split ratio, dividend amount
    pub adjustment_policy: AdjustmentPolicy,
    pub source: ProviderId,
    pub source_reference: String,
    pub parser_version: SemanticVersion,
}
```

ADJUSTMENT PROPAGATION

Action	Raw bars	Split-adj	Div-adj	Total-return
2:1 split	Unchanged	Price ÷ 2, volume × 2	Same as split-adj	Same as split-adj + dividend reinvestment
$0.50 dividend	Unchanged	Unchanged	Price − $0.50 on ex-date	Dividend reinvested at ex-date close
Delisting	Unchanged	Unchanged	Unchanged	Position removed at last available price

The backtest engine and chart system default to `TotalReturn` adjustment but expose the policy as a parameter. Every reproducibility manifest pins the adjustment policy used.

21. Symbology Resolution

RUST

```rust
/// Resolves user input ("AAPL", "apple inc", a CUSIP, a FIGI)
/// to a canonical asset ID. Handles ambiguity, delistings, and
/// ticker changes over time.
pub struct SymbologyService {
    /// Tries to resolve a user query to exactly one canonical asset.
    /// Returns multiple candidates if ambiguous, requiring disambiguation.
    pub fn resolve(
        &self,
        query: &str,
        at: Option<OffsetDateTime>,  // None = current; Some = point in time
    ) -> ResolutionResult;
}

pub enum ResolutionResult {
    Unique(AssetId),
    Ambiguous(Vec<AssetCandidate>),    // user must pick
    NotFound,
    Delisted {
        asset_id: AssetId,
        last_valid_date: Date,
        successor: Option<AssetId>,    // if ticker changed
    },
}
```

Part X — Desktop Backup, Restore, and Migration

> **DESIGN PRINCIPLE** For a local-first application, the user's machine is the system of record. Backup and restore must be first-class.

22. Backup Bundle

RUST

```rust
/// A complete backup of the local-first store. Signed and encrypted.
/// Contains all local databases, the vault (rewrapped to a backup key),
/// configuration, and a manifest of contents.
pub struct BackupBundle {
    pub manifest: BackupManifest,
    pub stores: Vec<StoreBackup>,       // SQLite, DuckDB, Parquet dirs
    pub vault: VaultBackup,             // rewrapped, not re-encrypted
    pub config: ConfigBackup,
    pub signature: BackupSignature,
}

pub struct BackupManifest {
    pub app_version: SemanticVersion,
    pub schema_versions: HashMap<StoreId, SemanticVersion>,
    pub created_at: OffsetDateTime,
    pub total_size_bytes: u64,
    pub item_counts: HashMap<String, u64>,  // {"strategies": 42, "journal_entries": 318}
    pub encryption: EncryptionInfo,
}
```

RESTORE PROTOCOL

80. User provides backup file + recovery key (or OS keychain unlock).
81. App verifies backup signature.
82. App checks schema versions against current app version.
83. If schemas differ, run migration sequence (old → current).
84. Restore stores to local paths.
85. Restore vault (rewrap from backup key to device key).
86. Validate integrity (record counts, hash checks).
87. App restarts with restored data.

CROSS-VERSION MIGRATION

Every local store (SQLite, DuckDB, Parquet layouts) has a `schema_version` table. On app launch, the migration runner checks the stored version against the app's expected version and runs sequential migrations (N → N+1 → N+2 → current). Migrations are signed and idempotent. If a migration fails partway, the runner rolls back to the pre-migration state and surfaces the error — never leaves the store in a half-migrated state.

Closing

This evolution document fills the gaps that would block or degrade implementation of the v0.1 overview. The priority order for implementation is:

88. **Part I — Trait Surfaces** (Strategy, Risk, BrokerGateway, ToolGateway, AlertEvaluator, Repository) — these are the architectural contract. Nothing else can be built correctly until these exist as reviewed ADRs.
89. **Part II — Pipeline Orchestration** — the data pipeline is the backbone of evidence-chained intelligence.
90. **Part III — Cost Model** — required for the Phase 0 exit criteria.
91. **Part IV — First-Run and Suitability** — determines whether the product's pitch lands.
92. **Parts V–X** — fill the remaining gaps as phases progress.

The v0.1 overview remains the authoritative product and security document. This document is its build-ready companion.

MYTHOS SYSTEMS / PRISMATIK

09

APPENDICES & OPERATING DOCTRINE

Glossary, architecture index, engineering mandate, document lineage, and implementation readiness controls.

CORNERSTONE EXECUTIVE ARCHITECTURE REPORT | 2026

Engineering Operating Doctrine

PRIME DIRECTIVE PRISMATIK is production software, not a demonstration. Every change must increase correctness, security, maintainability, observability, reproducibility, and operational confidence.

Correctness before cleverness

Encode invariants in domain types, validate every untrusted boundary, and prefer explicit behavior.

Security by default

Least privilege, safe defaults, abuse-case thinking, secret isolation, signed artifacts, and fail-closed controls.

Idempotency by design

Retries, redelivery, double-clicks, and uncertain network outcomes must not create duplicated trades or state.

Bounded concurrency

Every task, queue, scan, worker, and fan-out has ownership, cancellation, timeouts, limits, and backpressure.

Evidence and observability

Logs, metrics, traces, health, correlation IDs, provenance, and audit records explain important state transitions.

Reproducibility

Builds, datasets, models, strategies, simulations, migrations, and releases are versioned and repeatable.

AI as accelerator, not authority

AI output is untrusted until validated, tested, reviewed, and constrained by deterministic policy.

Cryptographic agility

Use strong modern protection now and preserve a documented migration path to hybrid and post-quantum profiles.

Minimum Definition of Done

- Requirements and acceptance criteria are explicit.
- Threat-model impact and abuse cases are considered.
- Contracts and migrations are versioned.
- Code formats, lints, compiles, and tests successfully.
- Failure paths and rollback are tested.
- Observability and runbooks exist.
- Secrets and sensitive data are protected.
- PQC posture and strongest-available fallback are documented.
- AI-generated portions are reviewed as untrusted.
- The project workbook is updated with actions, validation, risks, and continuation handoff.

Implementation Readiness Index

DOMAIN	LOAD-BEARING DELIVERABLE	TARGET
Product identity	Working name isolated from schemas and package contracts	Defined
Crypto foundation	CoinGecko provider adapter, identity service, cache, rate governor	Phase 0-1
Equity/options	Licensed quote feed and Unusual Whales adapter	Phase 2-3
Regulatory intelligence	SEC EDGAR and CFTC PRE pipelines	Phase 2
Quant runtime	Strategy IR, deterministic context, backtest, validation	Phase 4
Simulation	Bootstrap, parametric, jump, stochastic volatility, portfolio paths	Phase 5
AI controls	Tool gateway, read/draft permissions, no direct submit	Phase 1 onward
Execution	Broker gateway, risk policy, approval, idempotency, reconciliation	Phase 7

DOMAIN	LOAD-BEARING DELIVERABLE	TARGET
Security	Tauri boundaries, keychain/Vault, signed updates, audit, zero trust	Foundation
PQC agility	Crypto inventory, hybrid readiness, dual-sign manifests	Foundation onward
Enterprise	SSO, RBAC/ABAC, on-prem, SIEM, HSM/Vault	Phase 8
Ecosystem	Signed WASM plugins, MCP registry, review and revocation	Phase 9

Glossary of Core Terms

TERM	DEFINITION
ABAC	Attribute-Based Access Control; authorization based on attributes such as role, account, environment, device posture, and risk context.
ADR	Architecture Decision Record; a durable record of a significant technical decision, alternatives, consequences, security impact, and rollback plan.
Alpha	Return or value attributable to a strategy beyond its benchmark or modeled systematic exposure.
Backtest	A deterministic historical simulation of a strategy using point-in-time data and explicit execution assumptions.
CFTC	U.S. Commodity Futures Trading Commission.
COT	Commitments of Traders reports describing aggregated positioning in futures and options on futures.
CVaR	Conditional Value at Risk, also called expected shortfall; the expected loss in the tail beyond a VaR threshold.
Data lineage	The traceable chain from a derived value back to raw records, transformations, versions, and timestamps.
Delta	The sensitivity of an option price to a unit change in the underlying asset.
Gamma	The rate of change of delta as the underlying price changes.
Greeks	Option sensitivity measures including delta, gamma, theta, vega, and rho.
HSM	Hardware Security Module used to protect cryptographic keys and operations.
Idempotency	The property that safely repeating an operation produces the same intended final state instead of duplicates or corruption.
Implied volatility	The volatility level embedded in an option price under an option-pricing model.
IV rank	A comparison of current implied volatility with its historical range.
MCP	Model Context Protocol; a standard interface through which AI systems invoke external tools and data sources.
ML-DSA	NIST-standardized post-quantum digital signature algorithm derived from Dilithium.
ML-KEM	NIST-standardized post-quantum key-establishment mechanism derived from Kyber.

TERM	DEFINITION
Monte Carlo simulation	Repeated simulation of possible paths or trade sequences to estimate distributions of returns, drawdowns, and risk.
Opportunity	A ranked, evidence-backed market hypothesis - not a guaranteed trade instruction.
Order intent	A proposed trade expression that must pass deterministic risk, authorization, and approval controls before becoming an order.
PQC	Post-quantum cryptography designed to resist attacks from sufficiently capable quantum computers.
Provenance	Metadata identifying the source, endpoint, retrieval time, parser, transformation, and quality of data.
RBAC	Role-Based Access Control.
SEC EDGAR	The SEC filing and disclosure system used for company submissions and regulatory documents.
Sharpe ratio	A risk-adjusted return measure based on excess return divided by return volatility.
SLH-DSA	NIST-standardized stateless hash-based post-quantum signature algorithm.
Slippage	The difference between an assumed or quoted execution price and the realized fill price.
Sortino ratio	A risk-adjusted return measure that penalizes downside volatility rather than total volatility.
Strategy IR	A typed intermediate representation to which visual, DSL, Python-generated, or Rust strategies compile before execution.
Tauri	A Rust-centered framework for secure, cross-platform desktop applications using a web frontend.
Theta	The sensitivity of an option price to passage of time.
Value at Risk	A percentile estimate of potential loss over a defined horizon; insufficient as the sole measure of risk.
Vega	The sensitivity of an option price to implied volatility.
Walk-forward validation	Repeated train/validate/test windows that approximate how a strategy or model would have evolved through time.
WebGPU / wgpu	Cross-platform GPU graphics and compute technologies used for dense visualizations and selected parallel workloads.
Zero trust	A security model that continuously verifies identity, device, context, and authorization rather than assuming trust by network location.

Document Lineage and Stewardship

This cornerstone report integrates and preserves the complete product architecture from **PRISMATIK Overview v0.1** together with the build-ready contracts in **PRISMATIK Architecture Evolution v0.2**. It is intended to become the governing product and architecture reference until superseded through an explicit versioned decision.

VERSION	ROLE	STEWARDSHIP
v0.1	Product, experience, security, platform, and roadmap baseline	Preserved in Parts 1-7
v0.2	Load-bearing implementation contracts and missing architecture detail	Preserved in Part 8
Cornerstone 2026	Integrated executive and engineering reference	Mythos Systems / Aaron Stovall

STEWARDSHIP RULE Future revisions should preserve traceability to this document, record decisions in ADRs, update the architecture and security models, and maintain a continuation trail in /docs/workbook/.

MYTHOS SYSTEMS / PRISMATIK

BUILD THE INTELLIGENCE LAYER THE MARKET IS MISSING.

PRISMATIK is the secure, explainable, quantitative operating system that connects market evidence, institutional context, options intelligence, simulation, portfolio risk, controlled execution, and continuous learning.

Begin with a beautiful CoinGecko-powered crypto product. Preserve the architecture required for institutional options intelligence, credible quantitative research, secure execution, enterprise deployment, and a signed ecosystem. Build every phase as a complete vertical slice.

AARON STOVALL | MYTHOS SYSTEMS | 2026